



# 深度学习与神经网络

陈科海

计算机科学与技术学院  
哈尔滨工业大学（深圳）

[chenkehαι@hit.edu.cn](mailto:chenkehαι@hit.edu.cn)

# 目 录

01

神经网络与语言建模

02

循环神经网络模型

03

卷积神经网络模型

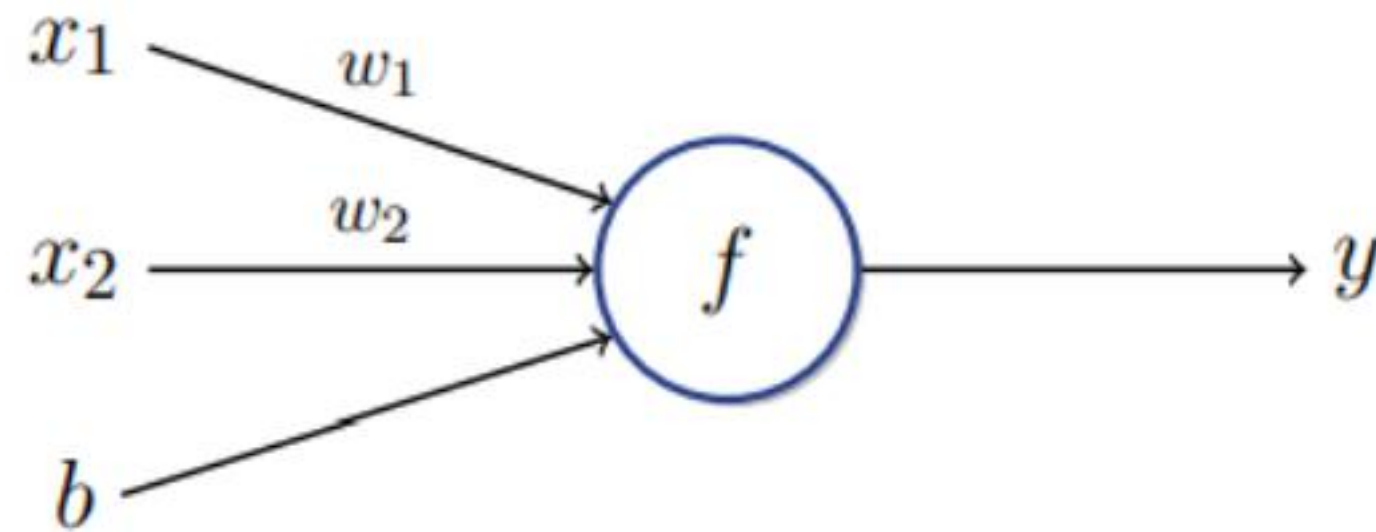
04

自注意力神经网络模型

# 1.神经网络和语言建模

人工神经元：人工神经网络的基本单元。

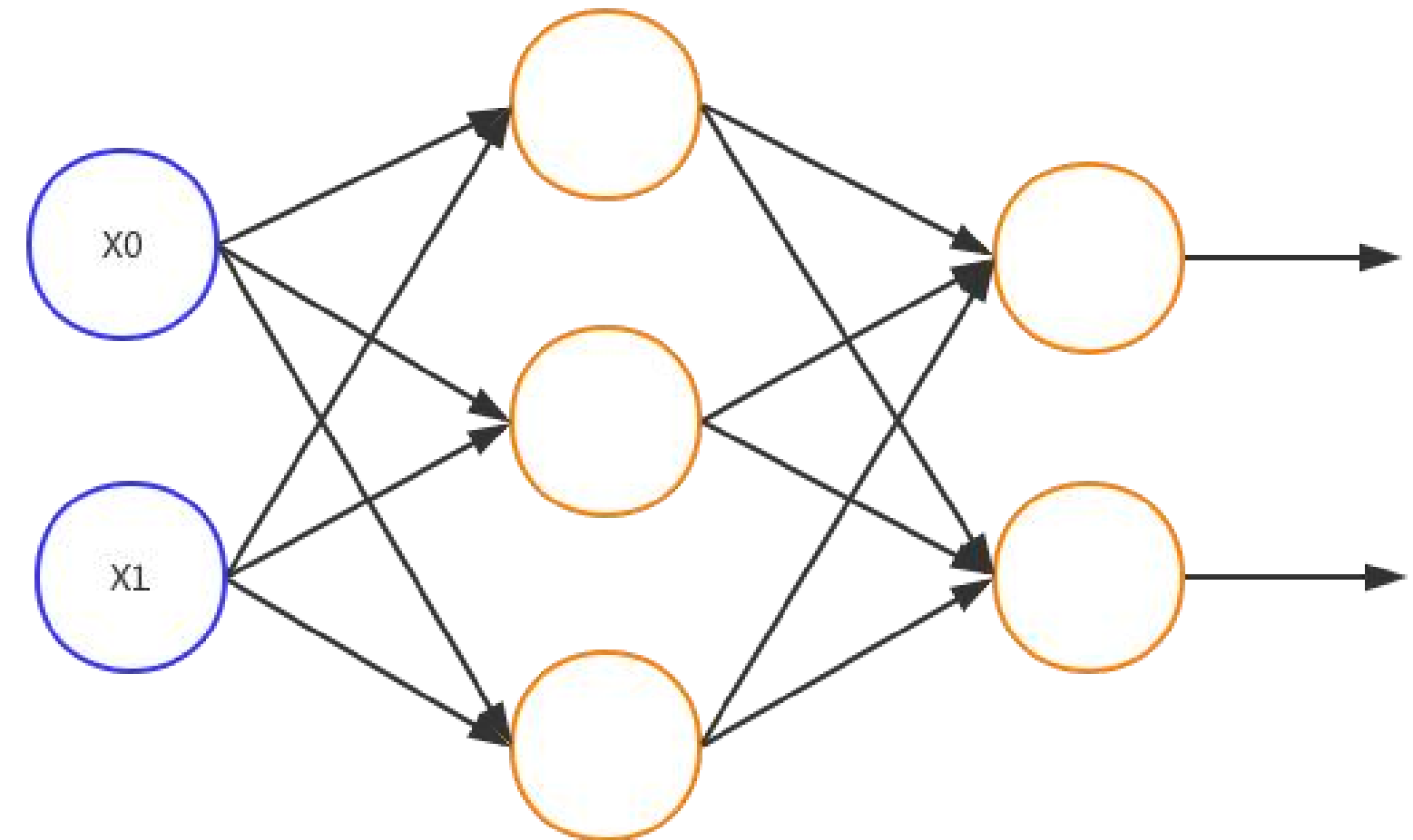
人工神经网络：描述客观世界的一种数学模型。



$$y = f(xw + b)$$

$x$ : 输入  
 $w$ : 权重  
 $b$ : 输出  
 $f$ : 激活函数  
 $y$ : 输出

线性  
非线性



# 1.神经网络和语言建模

实际问题主要涉及三方面问题：

对问题建模—— $x$

设计有效的决策模型—— $y$

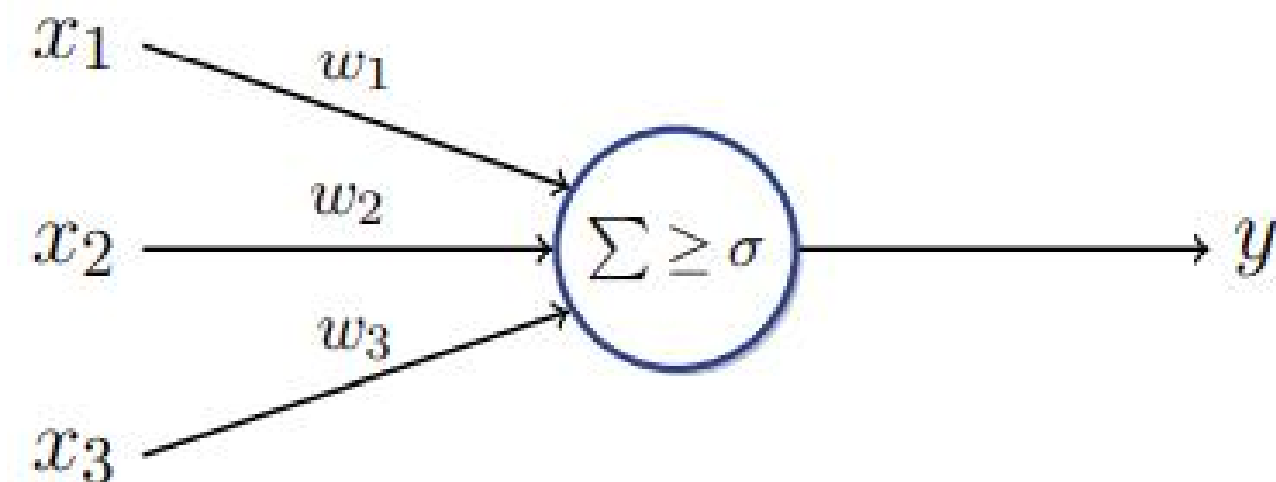
得到模型参数—— $w, b$

例如，是否去看电影？

$x$ : {电影院距离， 价格， 是否喜欢， ……}

$y$ : 去？ 不去？

感知机：最简单的人工神经元模型。



$$y = \begin{cases} 0 & \sum_i x_i \cdot w_i - \sigma < 0 \\ 1 & \sum_i x_i \cdot w_i - \sigma \geq 0 \end{cases}$$

$$x = (x_1, x_2, \dots, x_n)$$

$$w = (w_1, w_2, \dots, w_n)$$

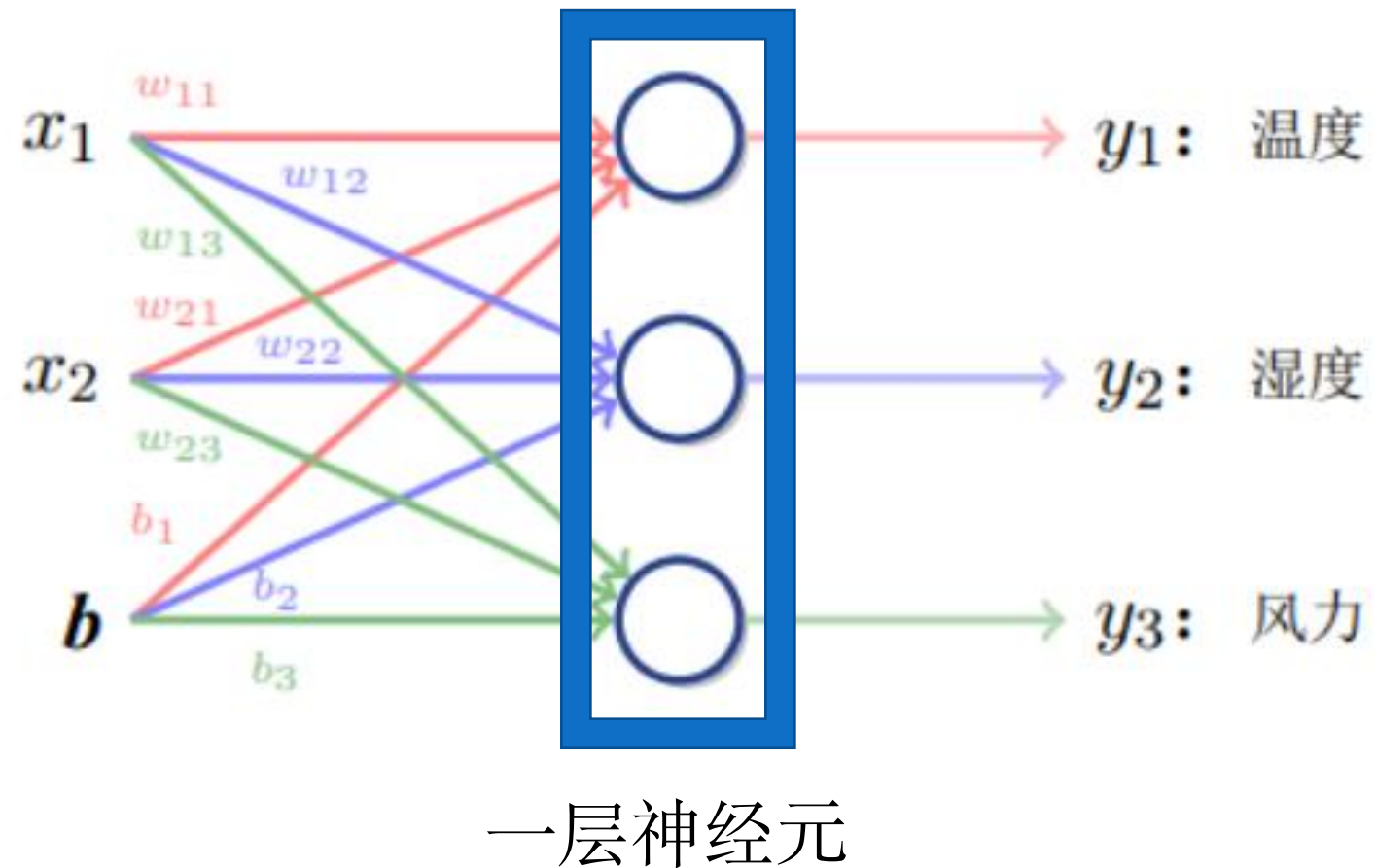
$$b = -\sigma$$

$$f = \Sigma \geq \sigma$$



# 1.神经网络和语言建模

## 单层神经网络



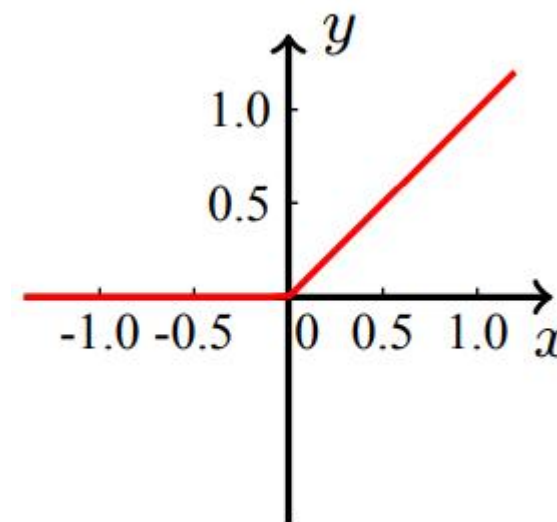
线性变换  $y = xW + b$

$$x = (x_1, x_2) \quad W = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix}$$

$$y = (y_1, y_2, y_3) \quad b = (b_1, b_2, b_3)$$

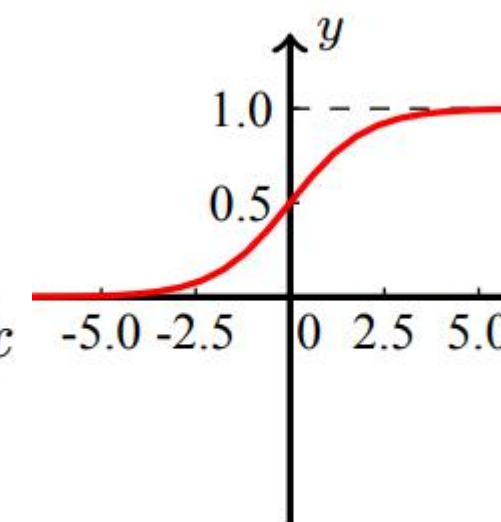
常见激活函数 Sigmoid ReLU Tanh

$$y = \max(0, x)$$



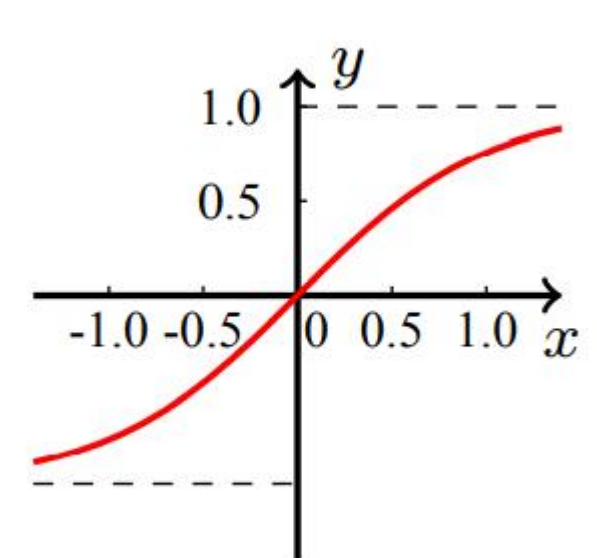
(a) ReLU

$$y = \frac{1}{1 + e^{-x}}$$



(b) Sigmoid

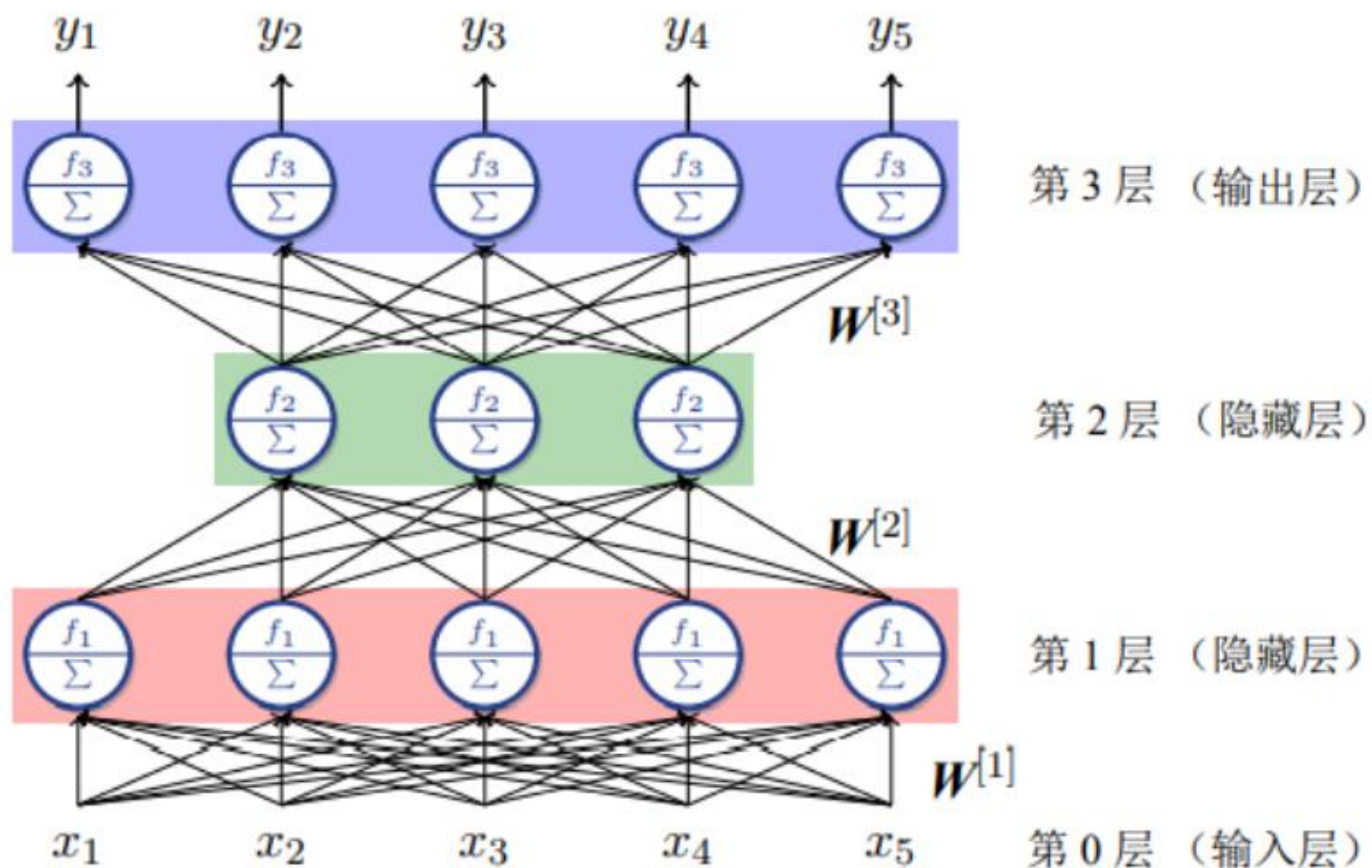
$$y = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



(c) Tanh

# 1.神经网络和语言建模

## 多层神经网络



# 1.神经网络和语言建模

## 前向传播

$$y = \text{Sigmoid}(\text{Tanh}(xW^{[1]} + b^{[1]})W^{[2]} + b^{[2]})$$

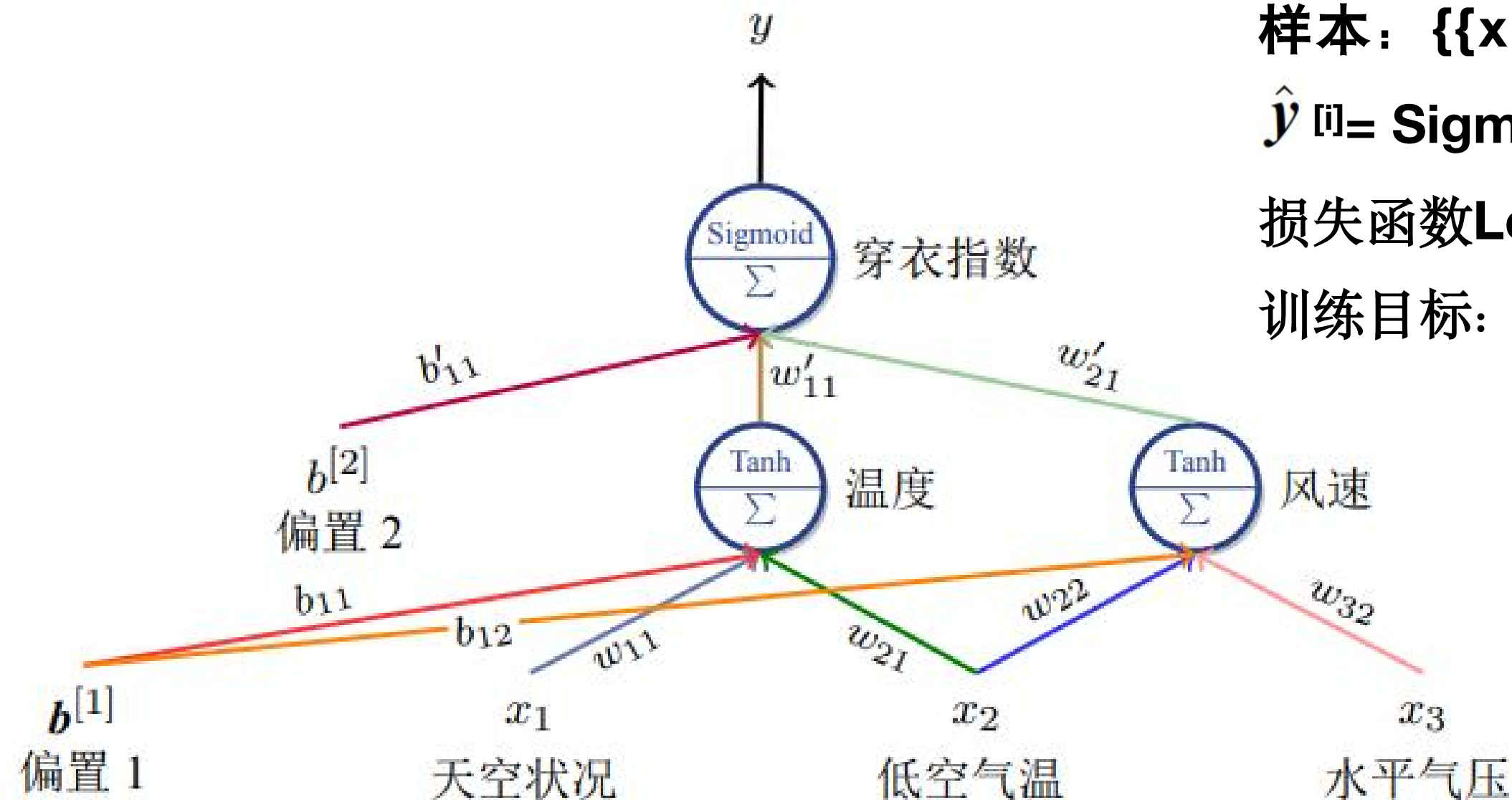
参数:  $W^{[1]}$ 、 $b^{[1]}$ 、 $W^{[2]}$ 、 $b^{[2]}$

样本:  $\{\{x^{[1]}, y^{[1]}\}, \{x^{[2]}, y^{[2]}\}, \dots\}$

$$\hat{y}^{[i]} = \text{Sigmoid}(\text{Tanh}(x^{[i]} W^{[1]} + b^{[1]})W^{[2]} + b^{[2]})$$

损失函数 $\text{Loss}(y, \hat{y}^{[i]})$ : 度量偏差

训练目标: 找到参数的最优值使损失函数最小



# 1.神经网络和语言建模

由于  $\hat{y}$  是由  $x$  和参数  $\theta$  决定的,  $\text{Loss}(y, \hat{y})$  也可以写做  $L(x, y, \theta)$

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n L(x^{[i]}, y^{[i]}; \theta)$$

优化目标: 找到  $\hat{\theta}$

$$\text{代价函数 } J(\theta) = \frac{1}{n} \sum_{i=1}^n L(x^{[i]}, y^{[i]}, \theta)$$

$$\text{参数优化方向 } \theta_{t+1} = \theta_t - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta}$$

$\alpha$ : 学习率

批量梯度下降: 数据规模大时, 每步迭代很慢

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L(x^{[i]}, y^{[i]}; \theta)$$

随机梯度下降: 可能收敛到局部最优

$$J(\theta) = L(x^{[i]}, y^{[i]}; \theta)$$

小批量梯度下降: batch\_size 的大小选择问

$$J(\theta) = \frac{1}{m} \sum_{i=j}^{j+m-1} L(x^{[i]}, y^{[i]}; \theta)$$



梯度消失：每层梯度都小于1，会导致梯度偏导数相乘很小，趋于0每次优化变化很小。

梯度爆炸：每层梯度都大于1，会导致梯度偏导数相乘很大，超出可表示范围。

梯度裁剪（解决梯度爆炸问题）：
$$\mathbf{g}' = \min\left(\frac{\sigma}{\|\mathbf{g}\|}, 1\right)\mathbf{g}$$

批量标准化：对神经网络隐藏层输出的每一个维度，沿着批次的方向进行均值为0、方差为1的标准化。

层标准化：对序列上同一层网络的输出结果，进行均值为0、方差为1的标准化。

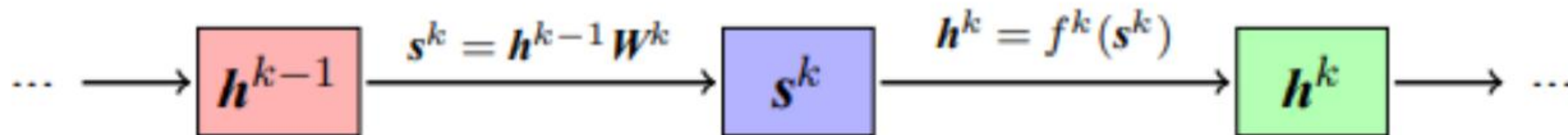
残差网络：
$$\mathbf{x}_{l+1} = F(\mathbf{x}_l) + \mathbf{x}_l$$

过拟合：模型过度拟合训练集，而对未见的测试集产生无法做出准确的判断。

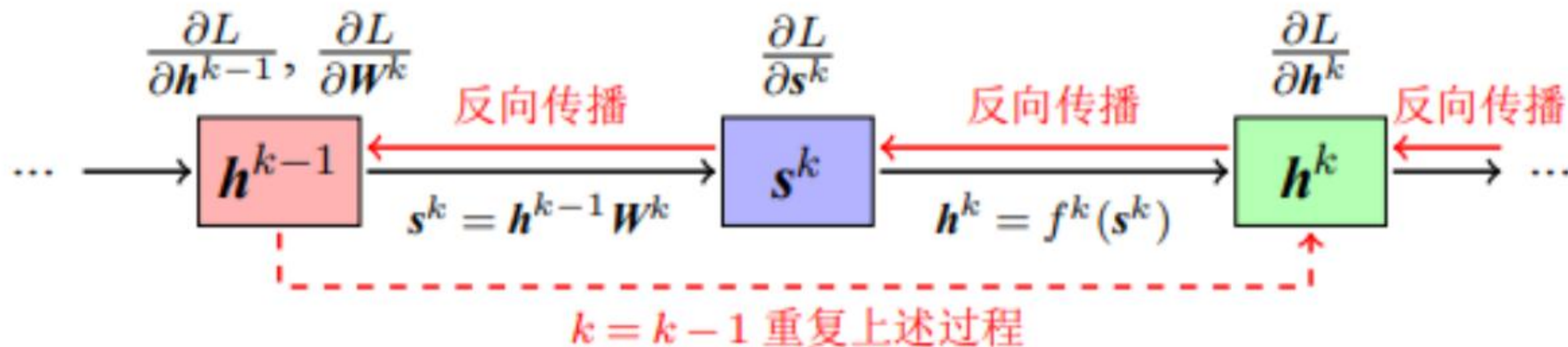
正则化：将目标函数变为 $J(\theta) + \lambda R(\theta)$ ， $R(\theta) = \sum_{i=1}^n \theta_i^2$

# 1.神经网络和语言建模

正向传播

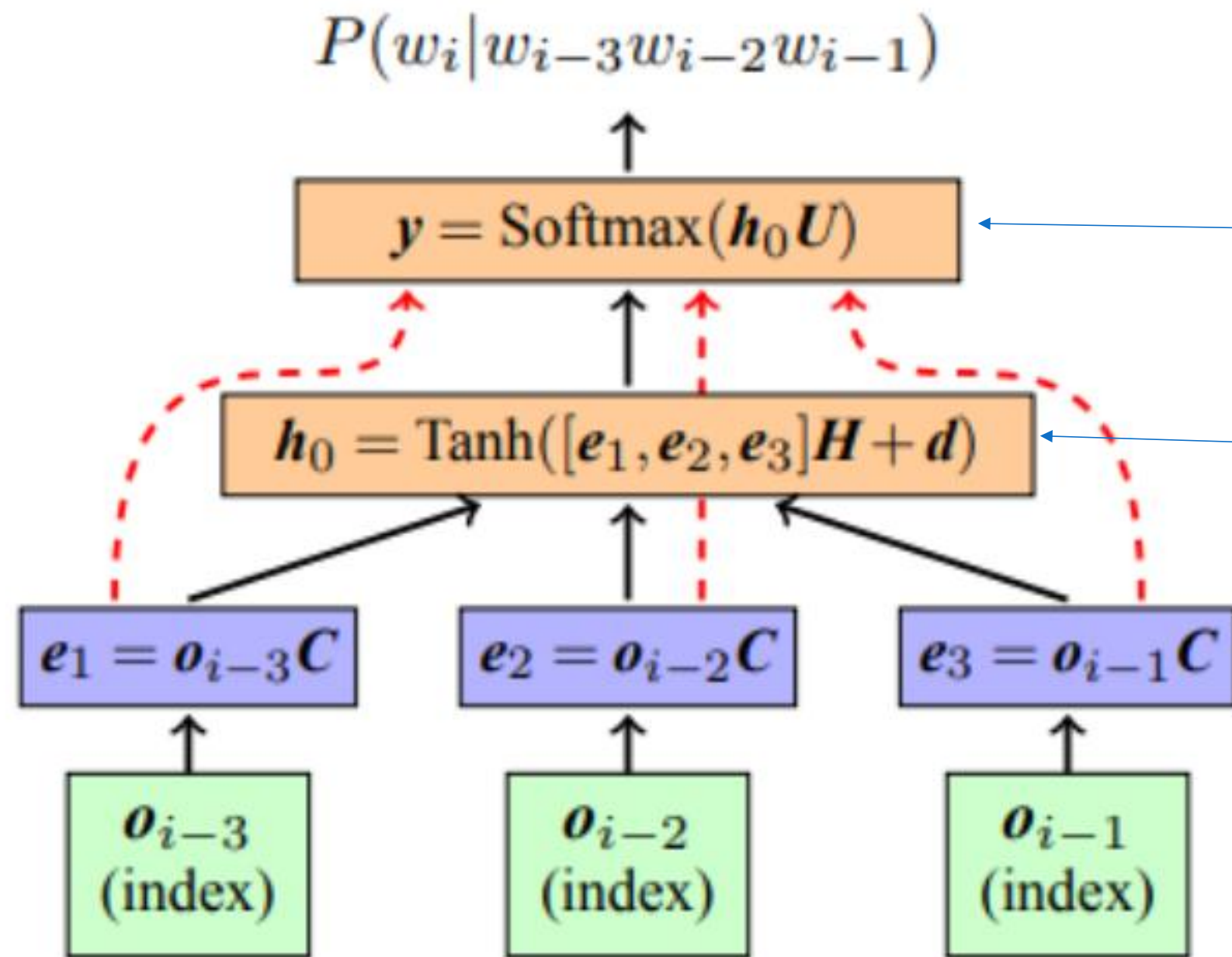


反向传播



# 1.神经网络和语言建模

## 基于前馈神经网络的语言模型 (4-gram为例)



问题:  $w_i$ 的生成概率只依赖之前的n-1个单词

输出层: 根据隐藏层输出预测单词概率分布

隐藏层: 将输入层的输出进行线性和非线性变换

输入层 (词嵌入): 将输入的One-hot离散向量转化为实数向量。

屋里/要/摆放/一个/\_\_\_\_  
屋里/要/摆放/一个/桌子  
屋里/要/摆放/一个/椅子

预测下个词  
见过  
没见过, 但是仍然是合理预测

# 目 录

01

神经网络与语言建模

02

循环神经网络模型

03

卷积神经网络模型

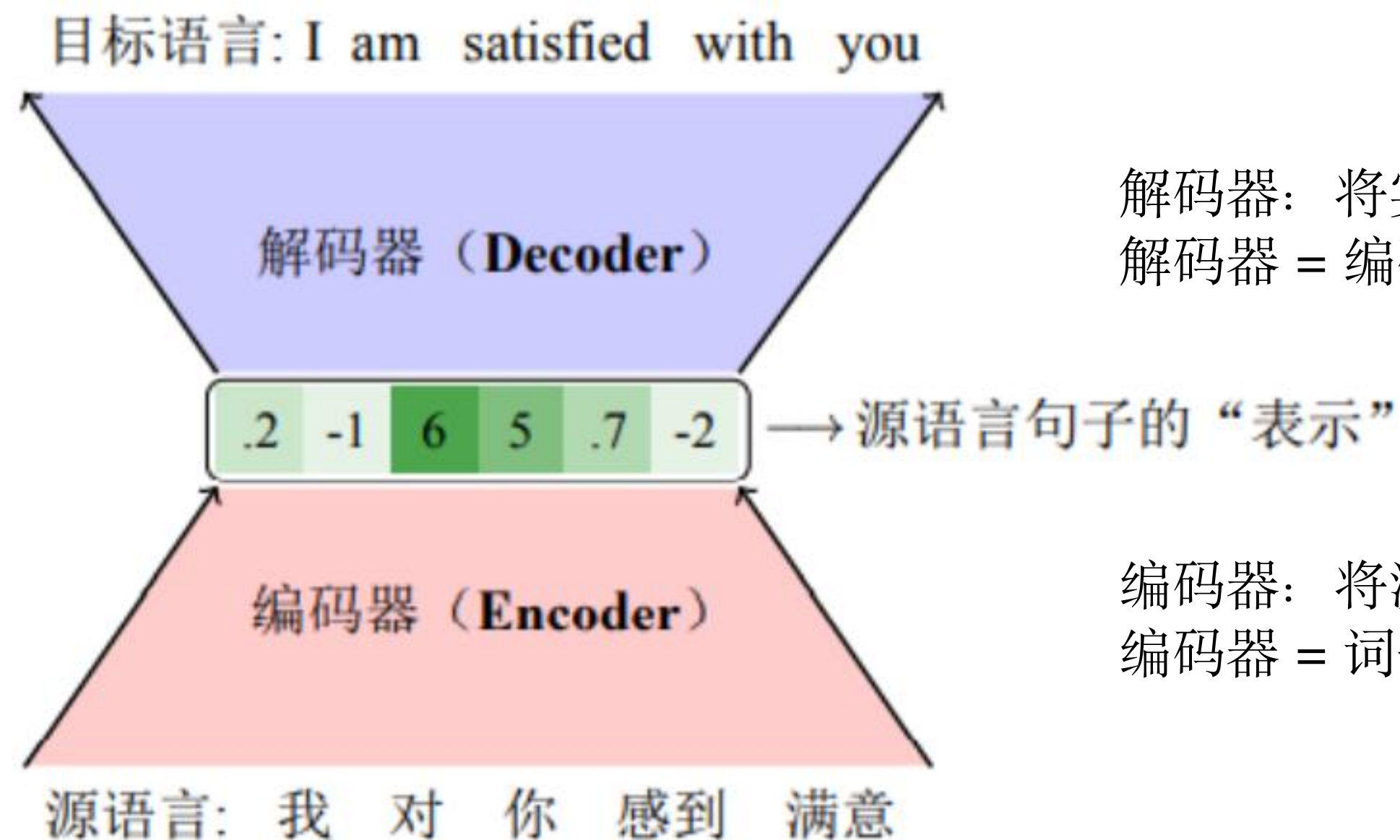
04

自注意力神经网络模型



## 2. 循环神经网络模型

### 编码器-解码器框架

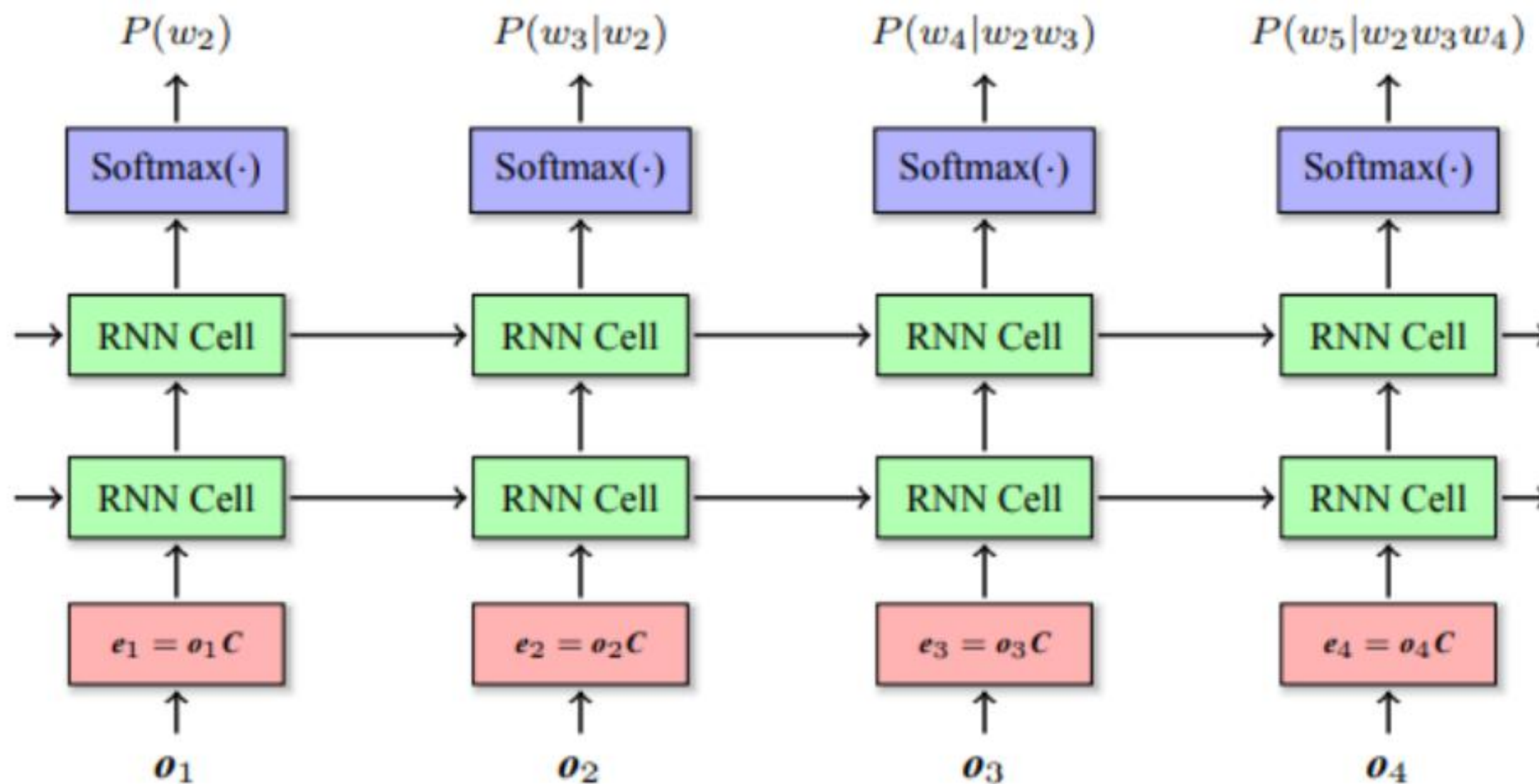


解码器: 将实数向量转换为目标语言  
解码器 = 编码器 + 输出层 (+ 编码-解码注意力子层)

编码器: 将源语言转换为实数向量  
编码器 = 词嵌入层 + 中间网络层

## 2.循环神经网络模型

循环神经网络的输入、输出都可以被看作是**时序序列**，每个时刻 $t$ 对应一个输入 $x_t$ 和输出 $y_t$ 。



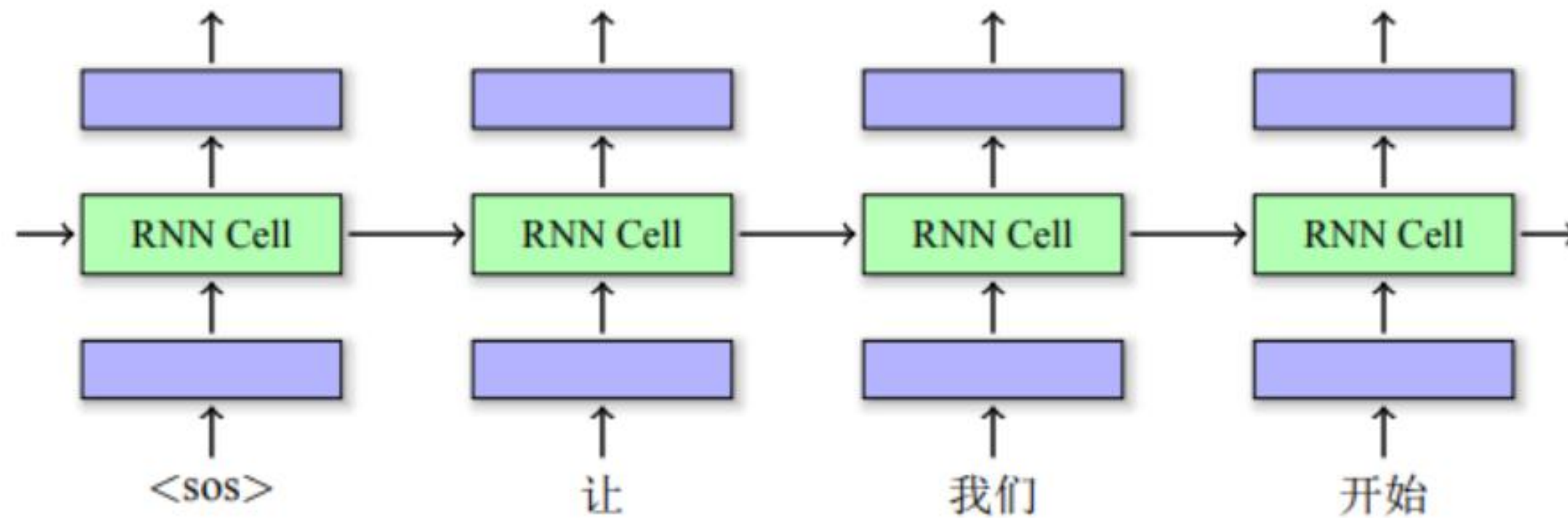
循环单元：读入前一时刻循环单元的输出  $h_{t-1}$  和当前时刻的输入  $x_t$ ，生成当前时刻循环单元的输出  $h_t$ 。

$$h_t = f(x_t U + h_{t-1} W + b)$$
$$y_t = \text{Softmax}(h_t V)$$

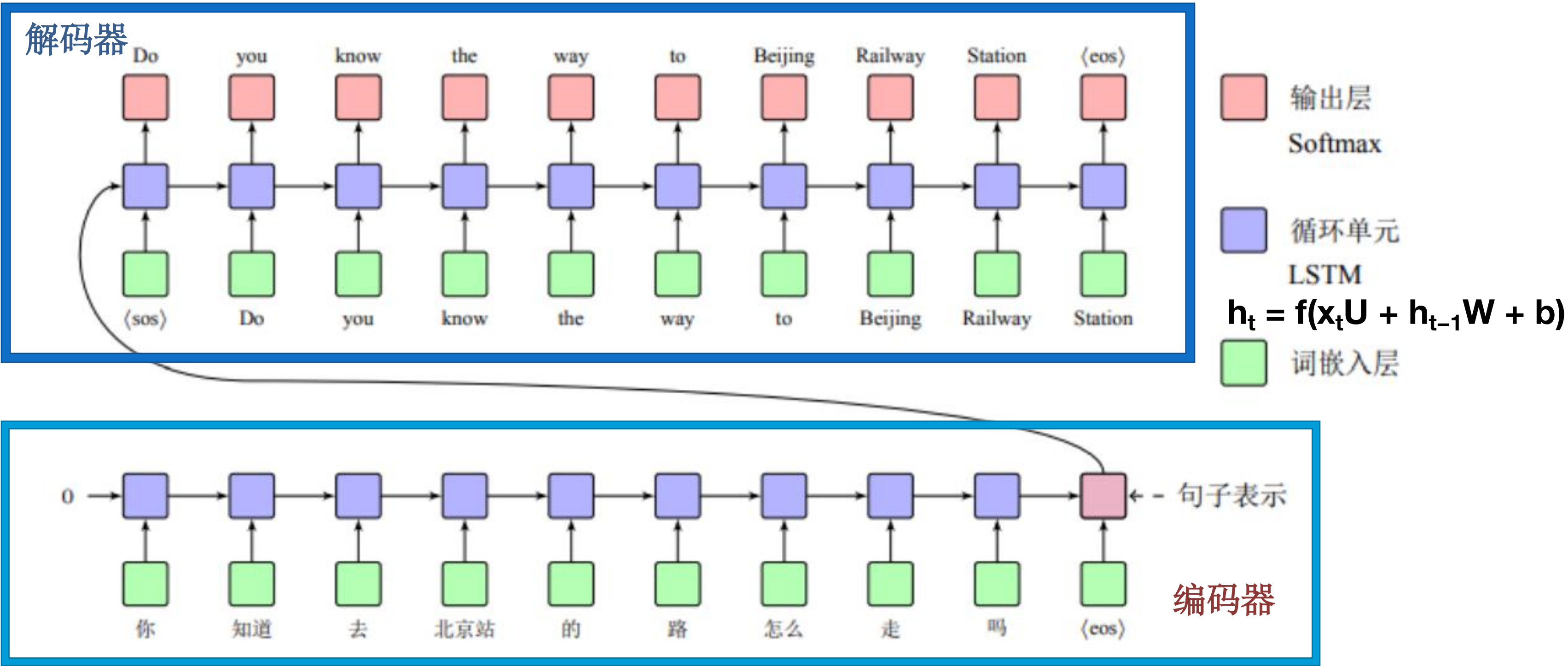
$U$ 、 $W$ 、 $b$ 、 $V$ 均为模型参数。

## 2.循环神经网络模型

模型对“开始”处理时，参考了“<sos> 让 我们”的信息。



# 2.循环神经网络模型



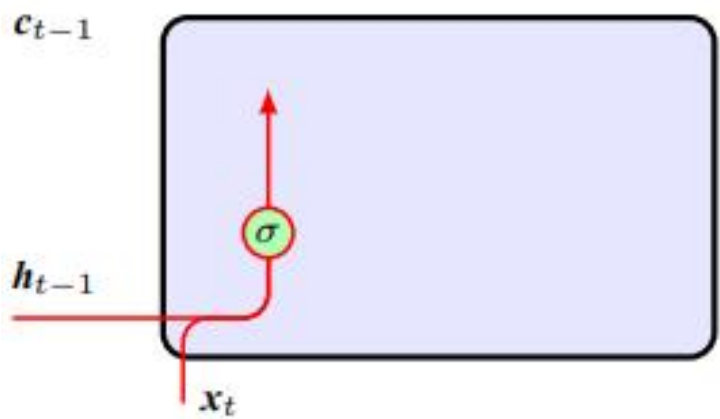


# 2.循环神经网络模型

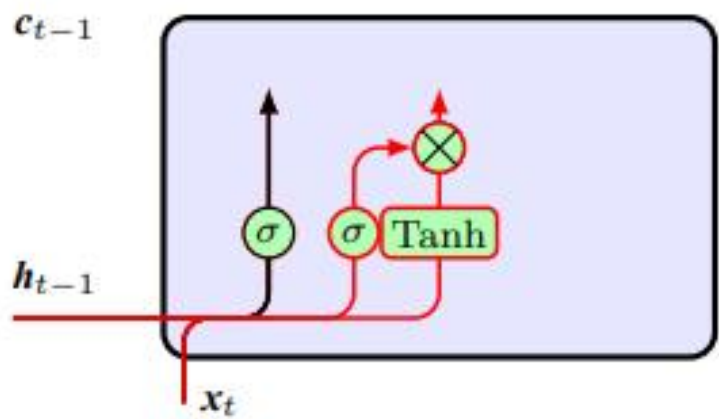
长短时记忆模型(LSTM): 同时传递两部分信息, 包括状态信息 $h_{t-1}$ 和记忆信息 $c_{t-1}$ 。  
结构包括遗忘、记忆更新、输出三部分。

遗忘门  
 $f_t = \sigma([h_{t-1}, x_t]W_f + b_f)$

遗忘 (保留) 部分先前信息



(a) 遗忘门



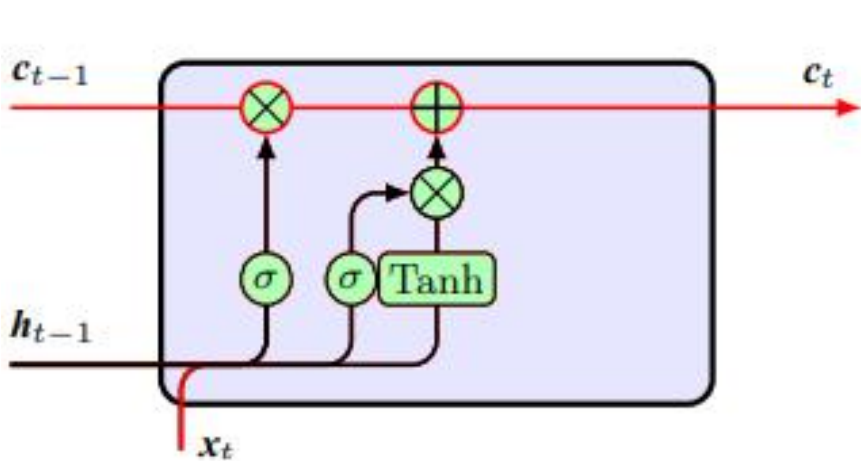
(b) 输入门

输入门  
 $i_t = \sigma([h_{t-1}, x_t]W_i + b_i)$   
 $\hat{c}_t = \text{Tanh}([h_{t-1}, x_t]W_c + b_c)$

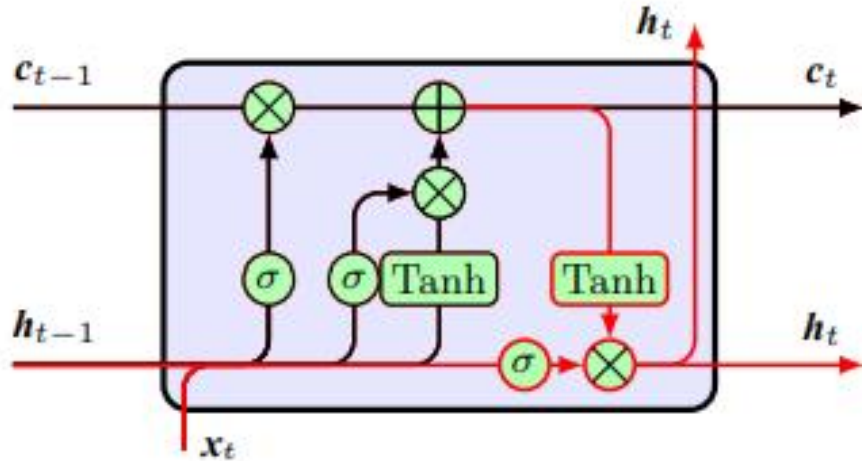
生成当前时刻新增信息

记忆更新  
 $c_t = f_t \odot c_{t-1} + i_t \odot \hat{c}_t$

结合两部分信息  
得到新记忆信息



(c) 记忆更新



(d) 输出门

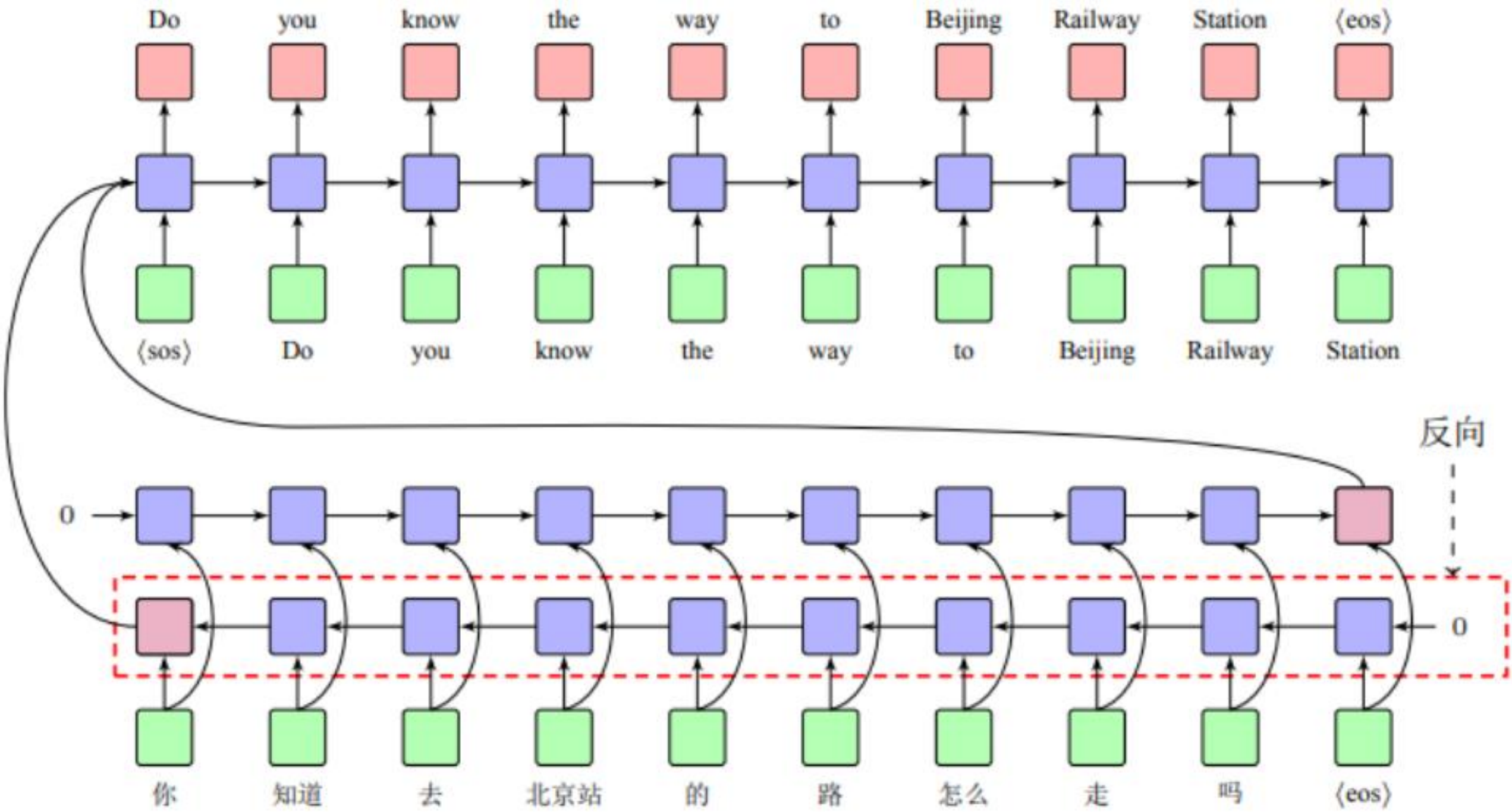
输出门  
 $o_t = \sigma([h_{t-1}, x_t]W_o + b_o)$   
 $h_t = o_t \odot \text{Tanh}(c_t)$

计算输出状态信息 $h_t$



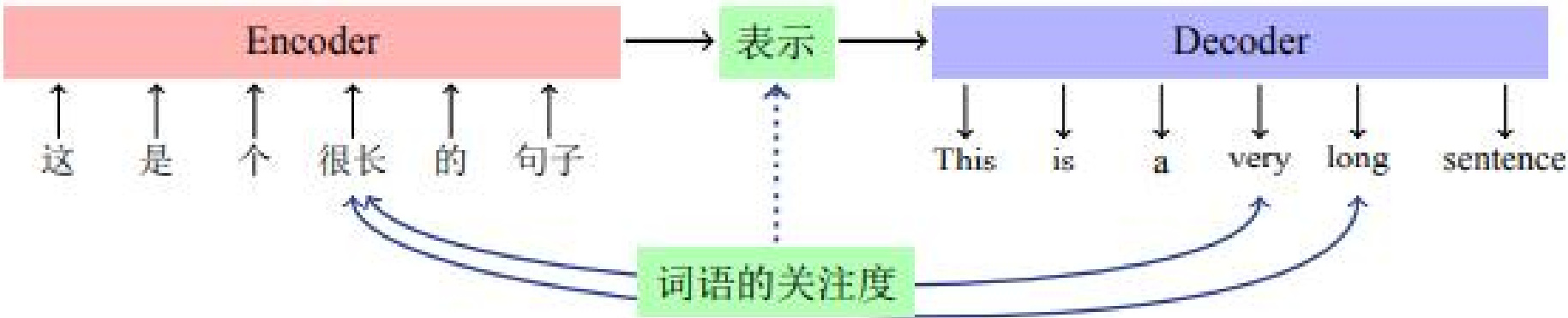
# 2.循环神经网络模型

双向模型：同时考虑上下文的信息。

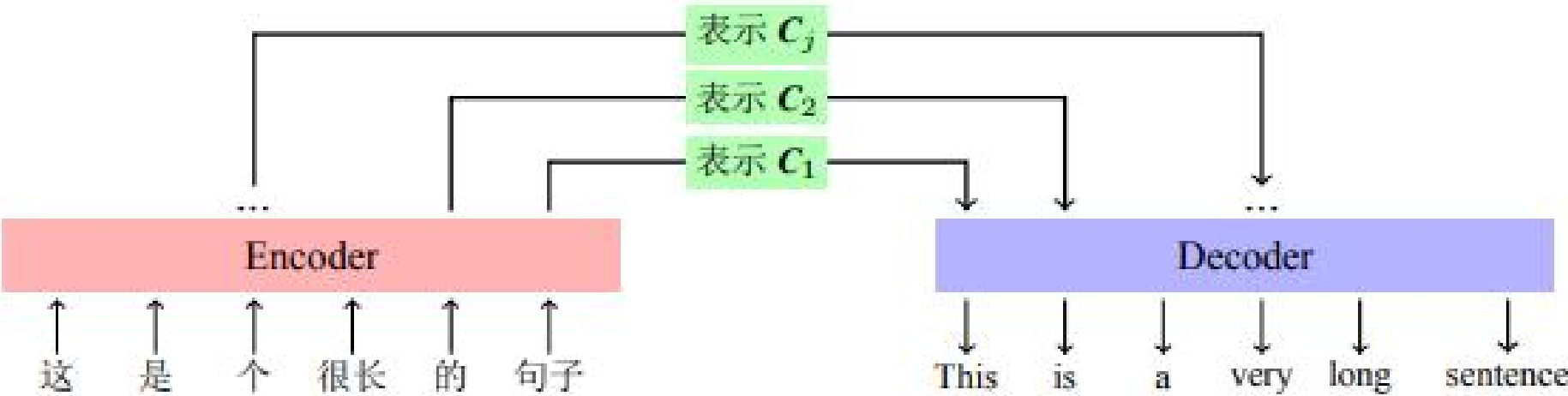


## 注意力机制

中午 没 吃饭， 又 打 了 一 下 午 篮 球， 我 现 在 很 饿， 我 想 吃 饭。



(a) 简单的编码器-解码器框架



(b) 引入注意力机制的编码器-解码器框架

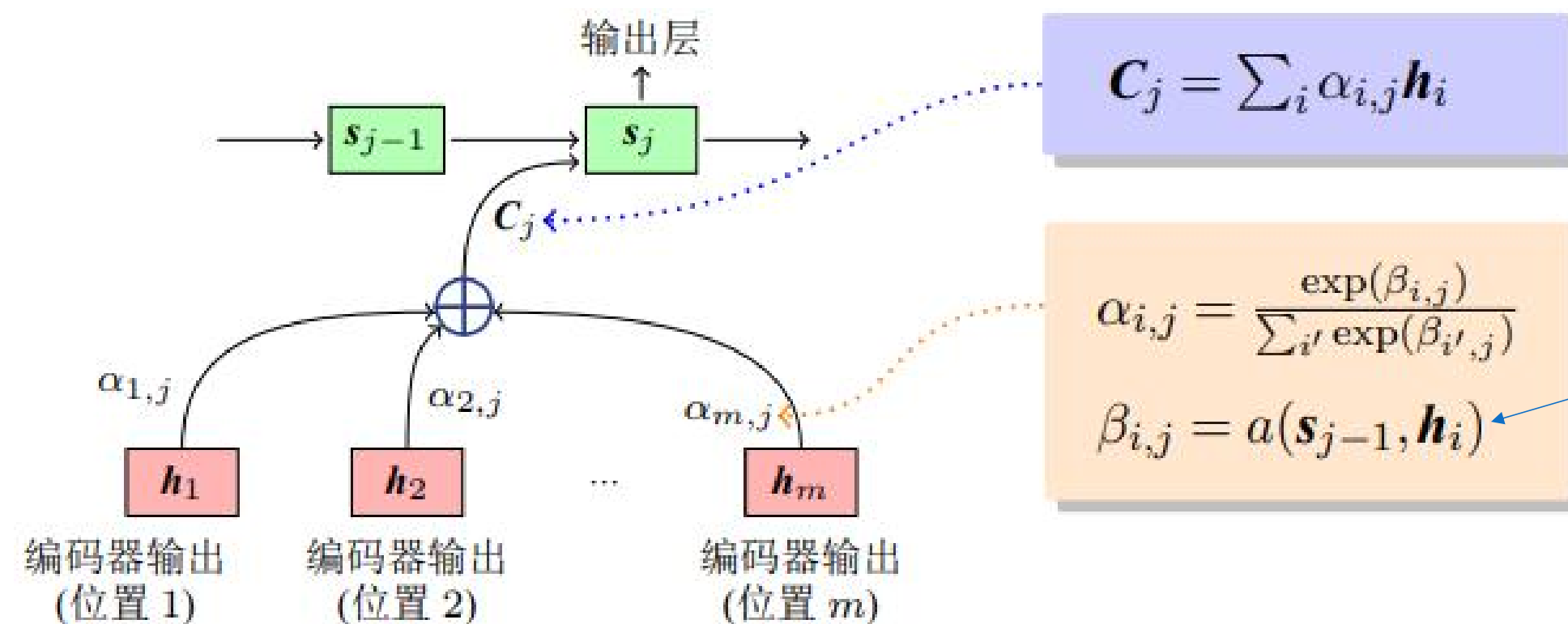
对于每个目标语言单词 $y_j$ ，系统生成一个源语言表示向量 $C_j$ 与之对应， $C_j$ 被称作对于目标语言位置 $j$ 的上下文向量。



## 2.循环神经网络模型

上下文向量的计算:  $C_j = \sum_i a_{i,j} h_i$

$a_{i,j}$  表示源语言第*i*个位置与目标语言第*j*个位置相关性大小



用目标语言前一时刻输出和源语言第*i*个位置输出的相关性, 来表示目标语言当前位置与源语言第*i*个位置相关性。

# 目 录

01

神经网络与语言建模

02

循环神经网络模型

03

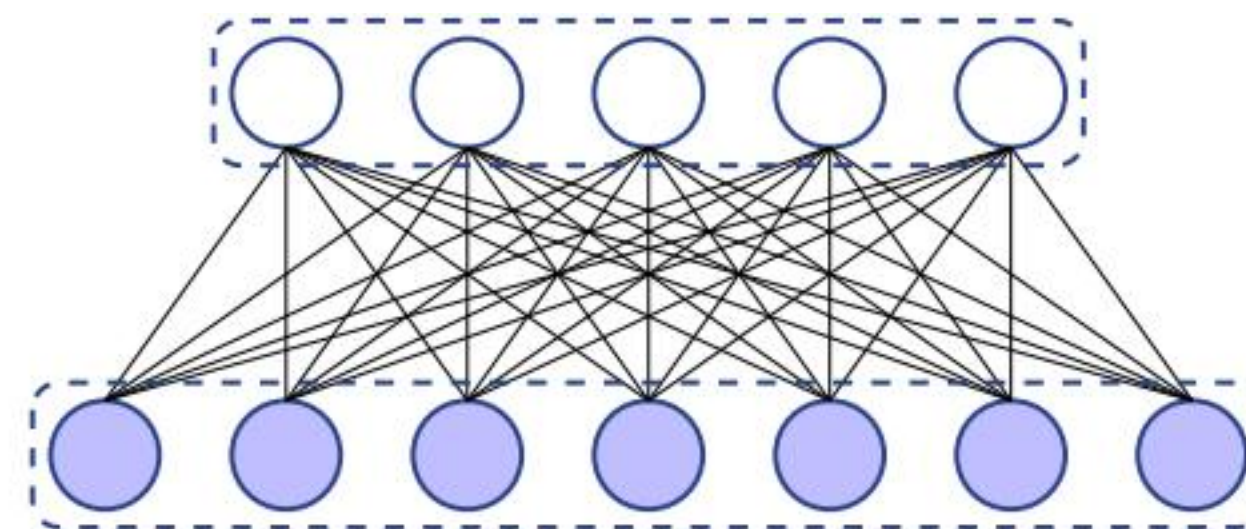
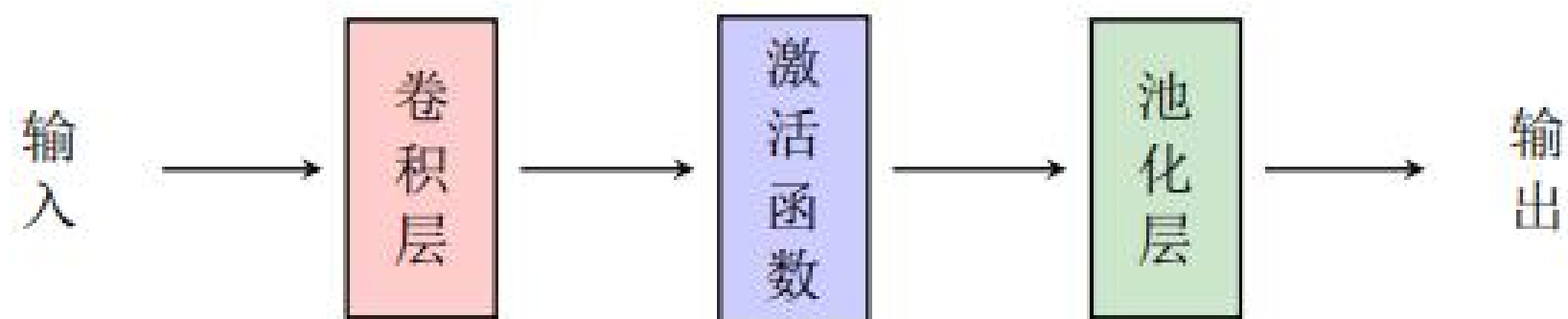
卷积神经网络模型

04

自注意力神经网络模型

### 3.卷积神经网络模型

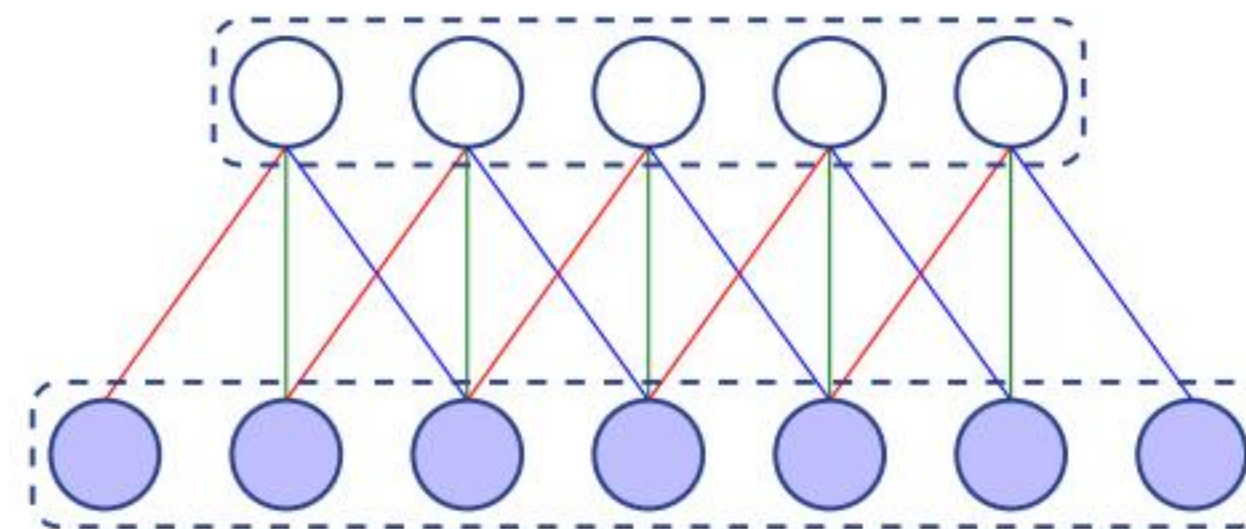
卷积神经网络：一种前馈神经网络，由若干的卷积层和池化层组成。



(a) 全连接层

特点：局部连接（减少连接数和参数量）

权值共享（减少参数量）

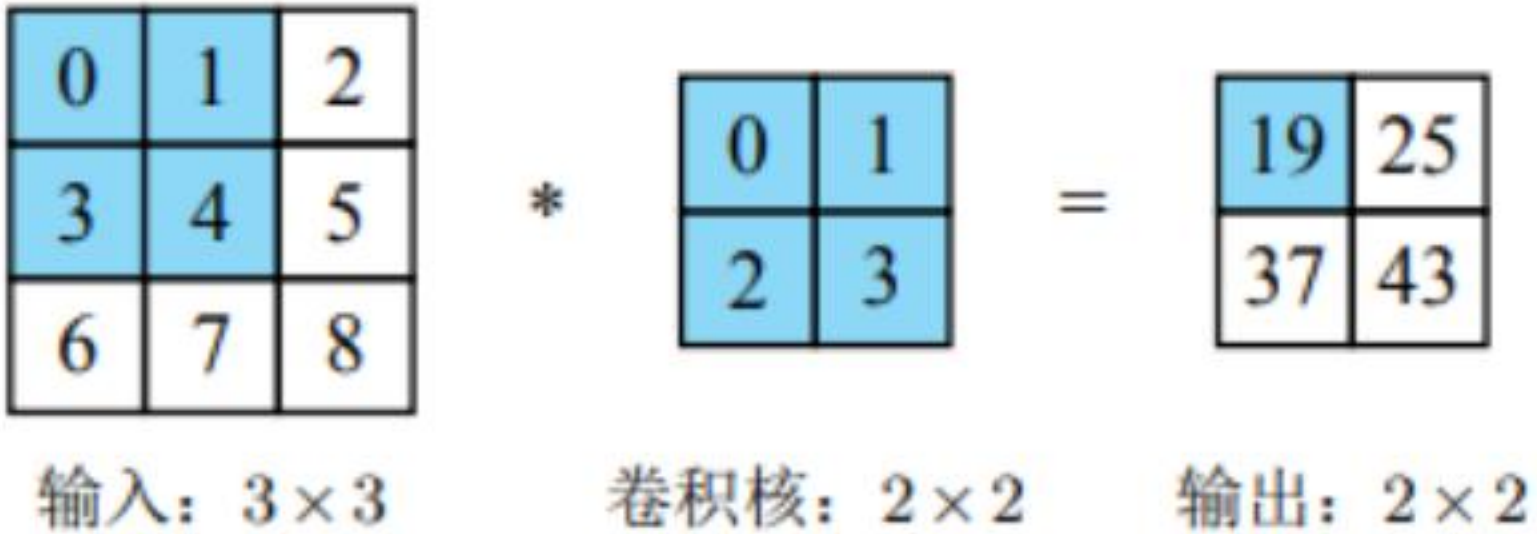
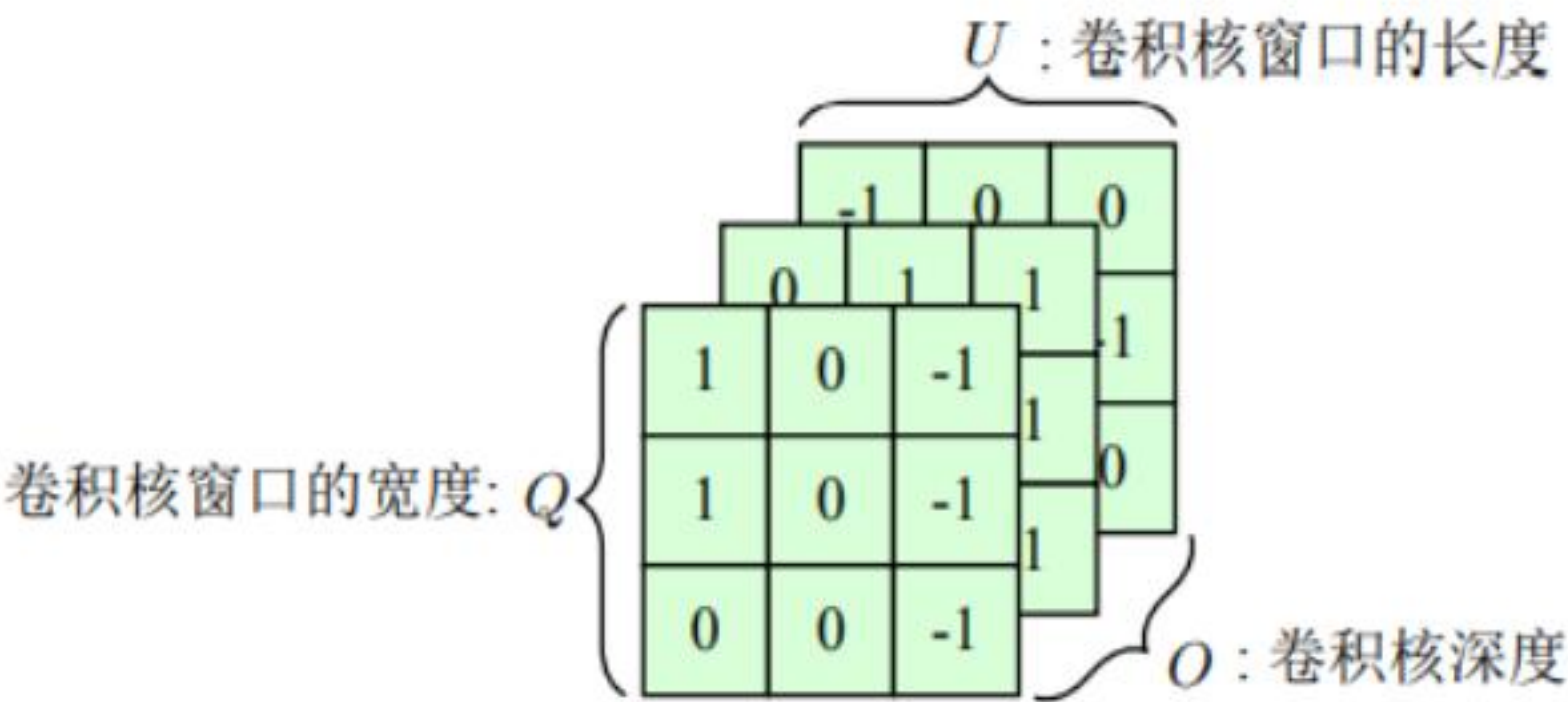


(b) 卷积层

# 3.卷积神经网络模型

卷积核：一组  $Q \times U \times O$  的参数，其中  $Q$  和  $U$  表示卷积核窗口的宽度与长度， $O$  是该卷积核的深度，它的取值和输入数据通道数保持一致。

$$\begin{aligned} y_{0,0} &= \sum \sum (x_{[0 \times 1:0 \times 1+2-1, 0 \times 1:0 \times 1+2-1]} \odot w) \\ &= \sum \sum (x_{[0:1,0:1]} \odot w) \\ &= \sum \sum \begin{pmatrix} 0 \times 0 & 1 \times 1 \\ 3 \times 2 & 4 \times 3 \end{pmatrix} \\ &= 0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 \\ &= 19 \end{aligned}$$





# 3.卷积神经网络模型

步长：卷积核每次滑动的距离。

- 问题： 1.边缘区域的点只会被计算一次。
- 2.多次卷积后维度不断减小，影响泛化能力。

解决： 填充

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 4 | 2 | 2 | 0 |
| 0 | 2 | 0 | 1 | 4 | 0 | 0 | 0 |
| 0 | 1 | 2 | 3 | 2 | 0 | 0 | 0 |
| 0 | 2 | 0 | 0 | 1 | 5 | 2 | 0 |
| 0 | 1 | 1 | 2 | 0 | 2 | 2 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

输入：8×8（填充后）

\*

|   |   |   |
|---|---|---|
| 0 | 1 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |

卷积核：3×3

=

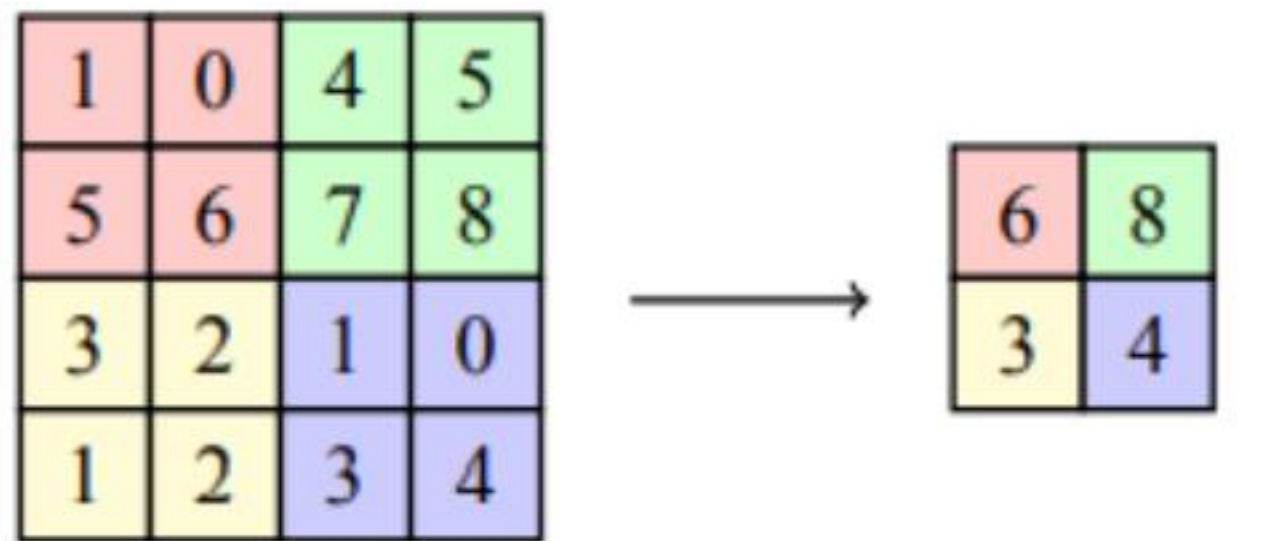
|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 2 | 5 | 2 | 4 |
| 2 | 1 | 3 | 8 | 7 | 3 |
| 2 | 7 | 9 | 9 | 8 | 2 |
| 4 | 2 | 7 | 8 | 7 | 2 |
| 4 | 8 | 7 | 2 | 3 | 7 |
| 2 | 1 | 2 | 8 | 7 | 4 |

输出：6×6

### 3.卷积神经网络模型

池化：根据设定的窗口进行滑动选取局部信息计算，无参数。

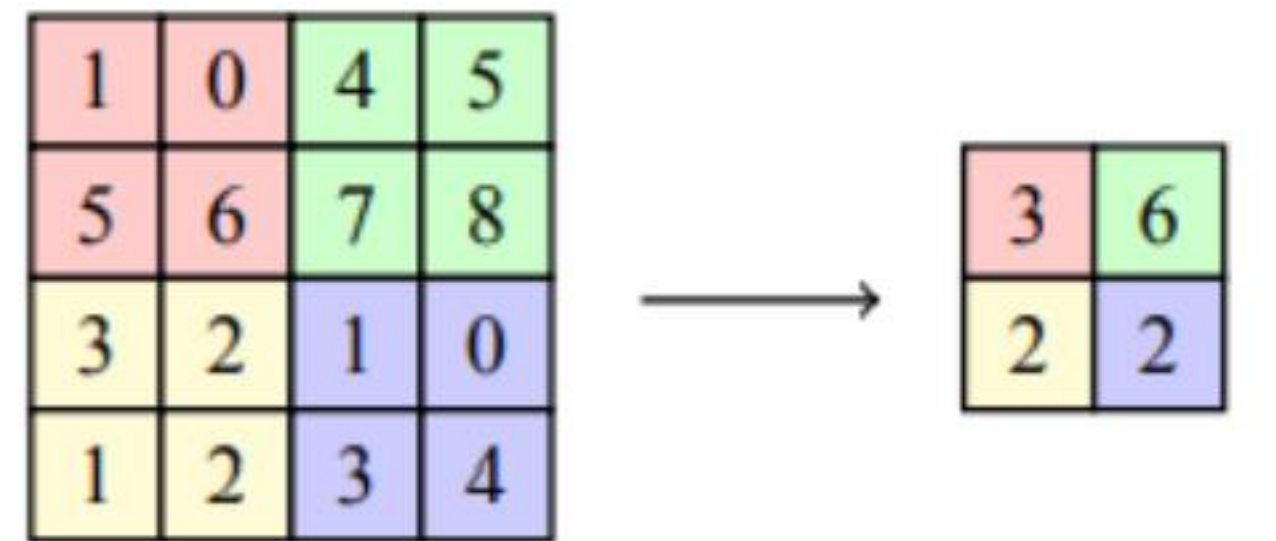
常见的池化有最大池化和平均池化。



输入：4×4

输出：2×2

(a) 最大池化



输入：4×4

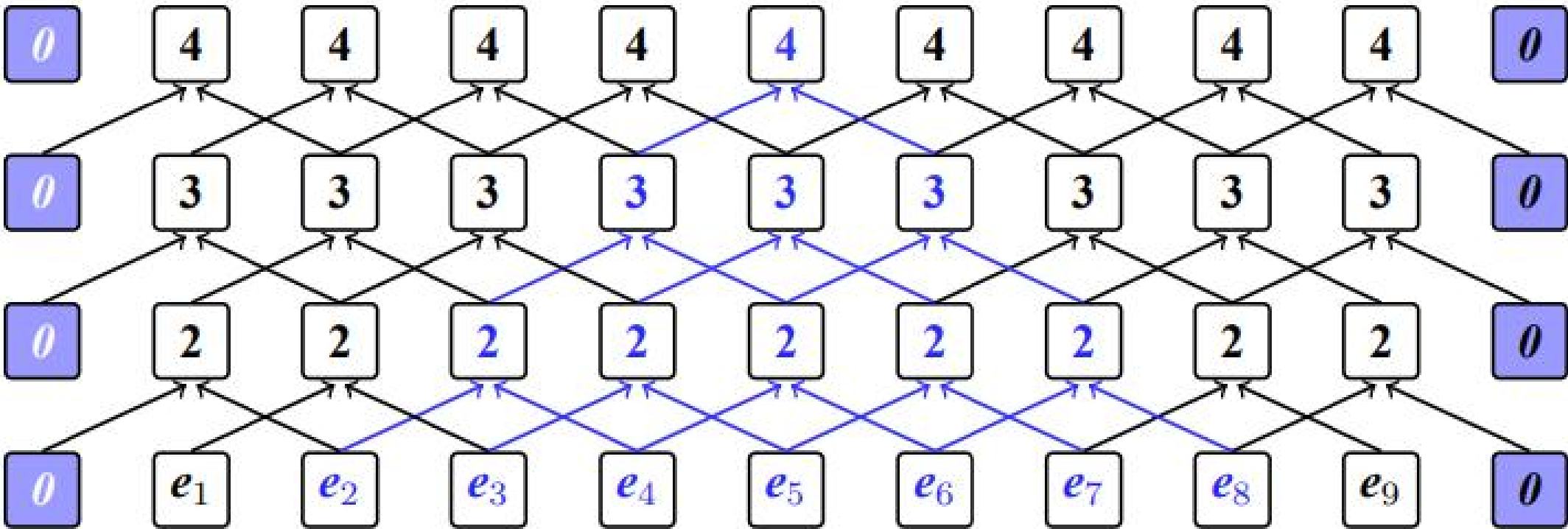
输出：2×2

(b) 平均池化

## 面向序列的卷积



(a) 循环神经网络的串行结构

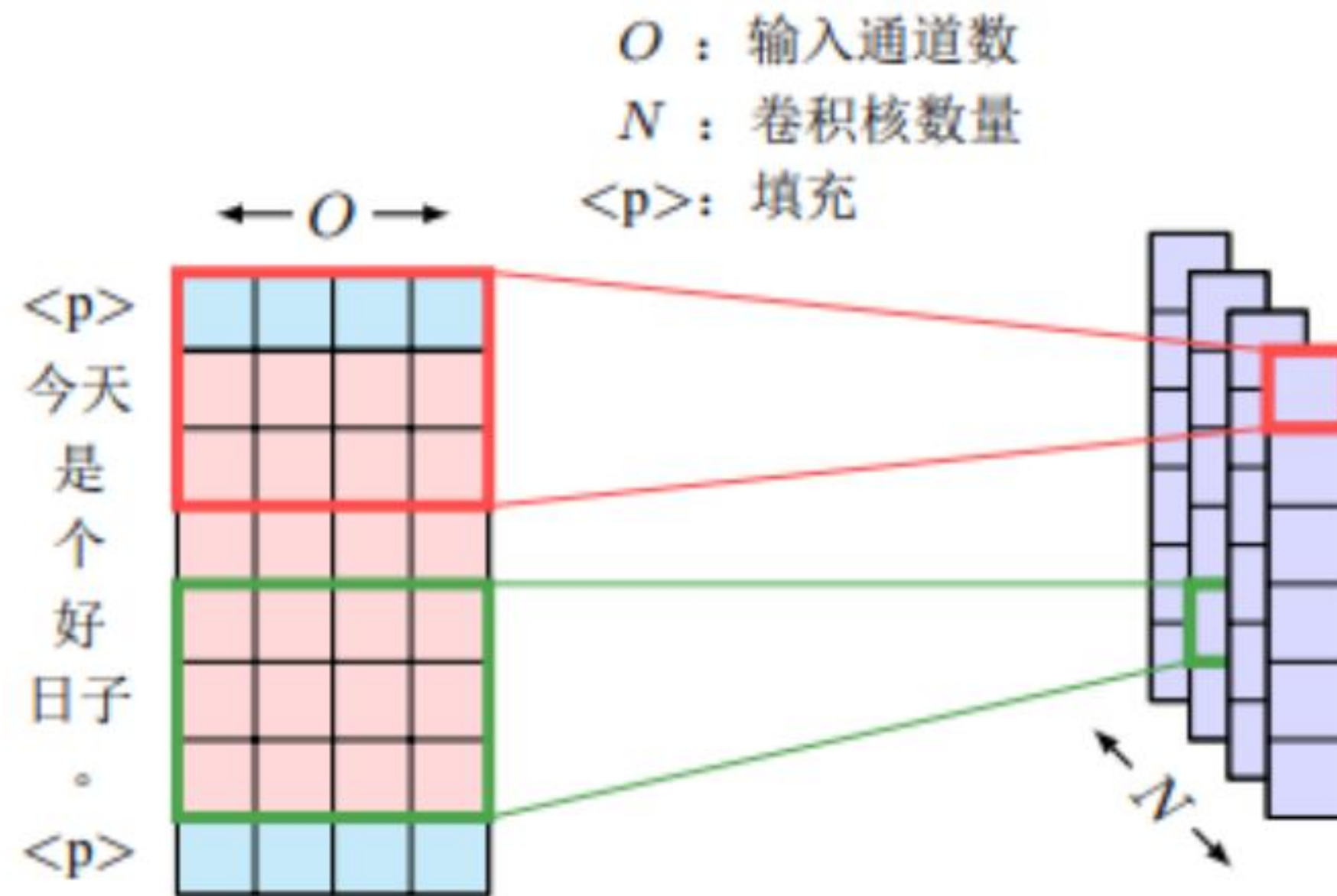


(b) 卷积神经网络的层级结构

### 3.卷积神经网络模型

卷积核大小为 $K$ , 通道数 $O$ =词嵌入表示维度, 卷积核数量 $N$

整体参数量为 $K \times O \times N$





### 3.卷积神经网络模型

#### 基于卷积神经网络的翻译模型

- ①每一层完全并行化，避免了计算顺序对时序的依赖
- ②层级结构可以捕捉不同位置之间的依赖
- ③通过门控线性单元、残差网络和位置编码等技术提升性能

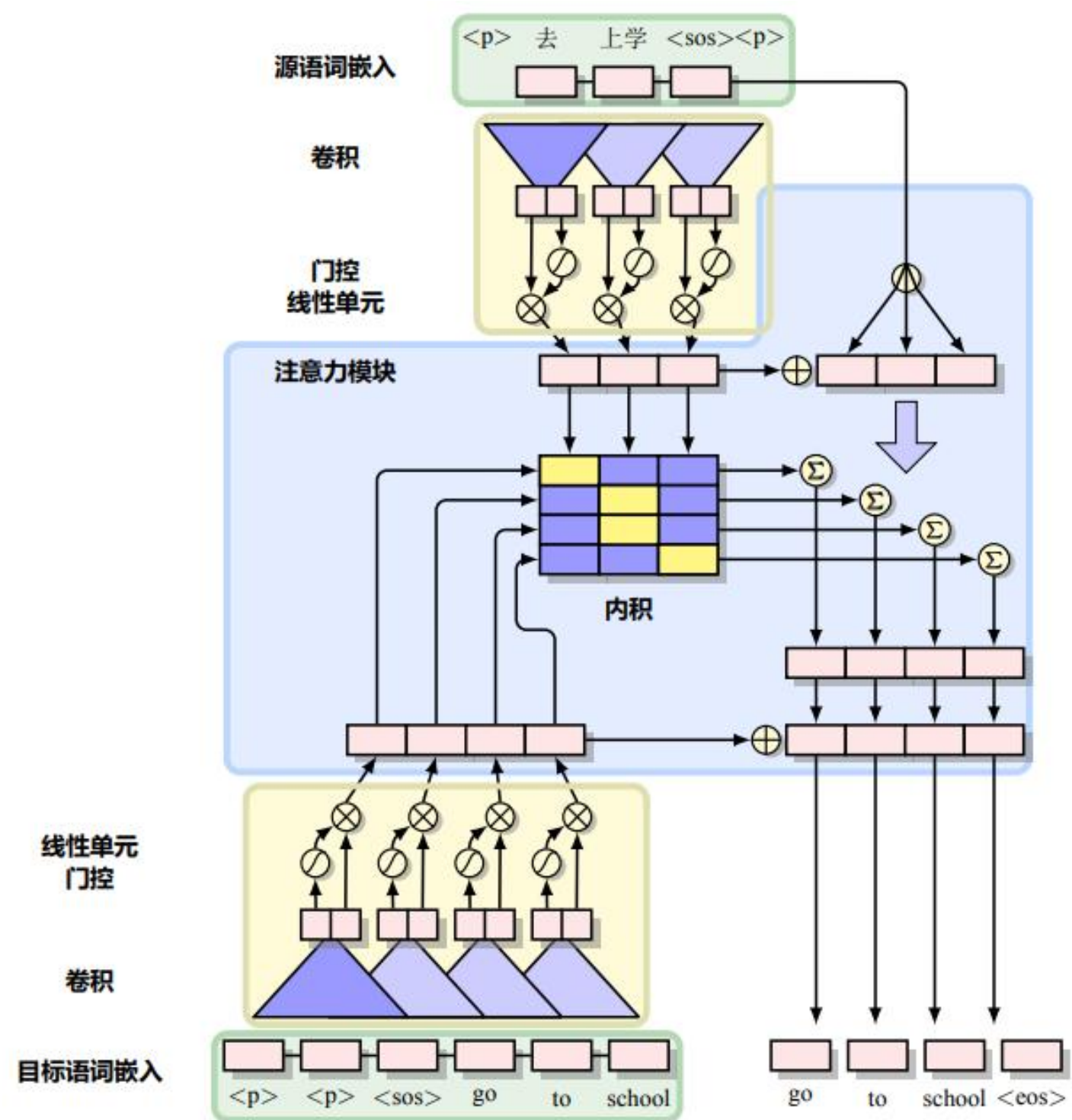


图 11.12 ConvS2S 模型结构

## 位置编码

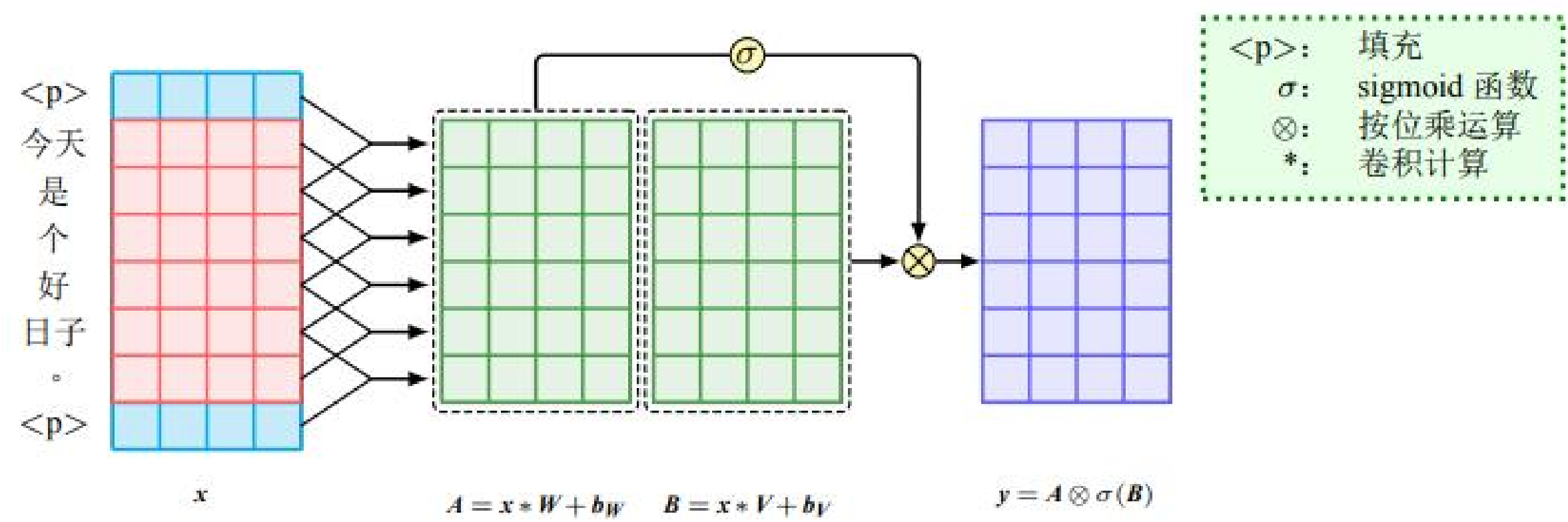
**问题:** 单层的卷积神经网络只能捕捉到局部的相对位置信息，即使多层也对全局位置表示不充分。

引入位置编码  $\mathbf{p} = \{\mathbf{p}_1, \dots, \mathbf{p}_m\}$ ，其中  $\mathbf{p}_i$  的维度大小为  $d$ ，一般和词嵌入维度相等。

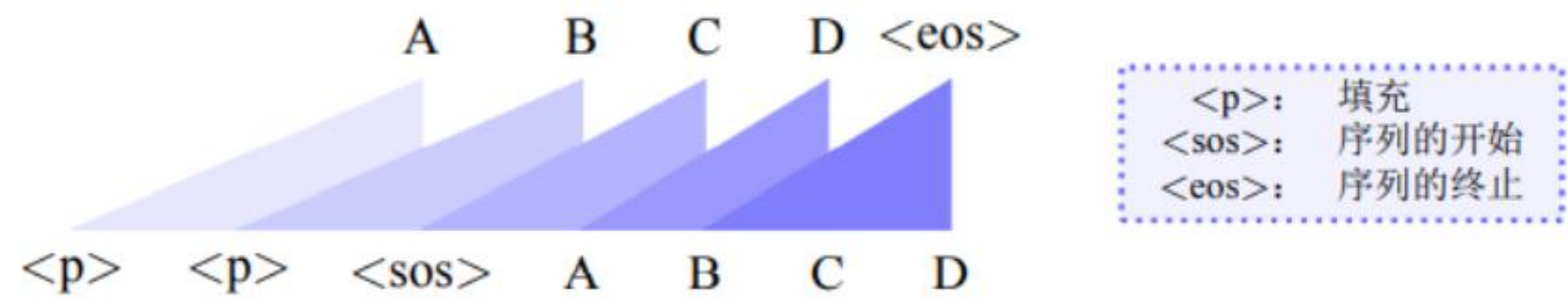
模型的输入序列可将词嵌入矩阵与位置编码相加，即  $\mathbf{e} = \{\mathbf{e}_1, \dots, \mathbf{e}_m\} = \{\mathbf{w}_1 + \mathbf{p}_1, \dots, \mathbf{w}_m + \mathbf{p}_m\}$

# 3.卷积神经网络模型

## 门控卷积神经网络



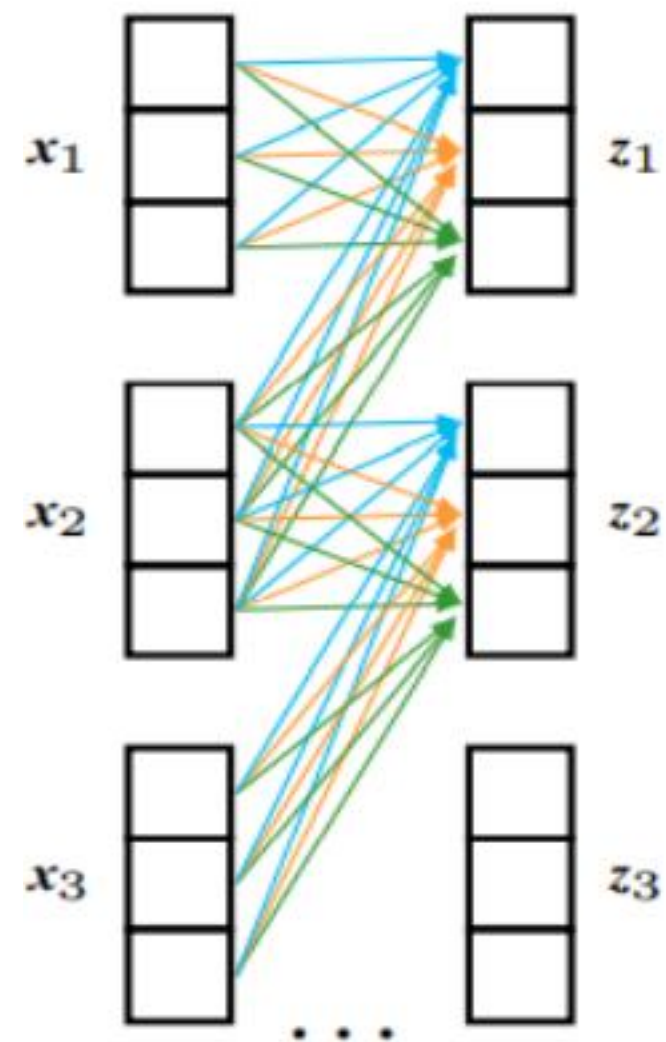
解码器中由于要屏蔽未来的信息，需要在头部填充K-1的空元素，K为卷积核大小



# 3.卷积神经网络模型

深度可分离卷积：深度卷积和逐点卷积结合

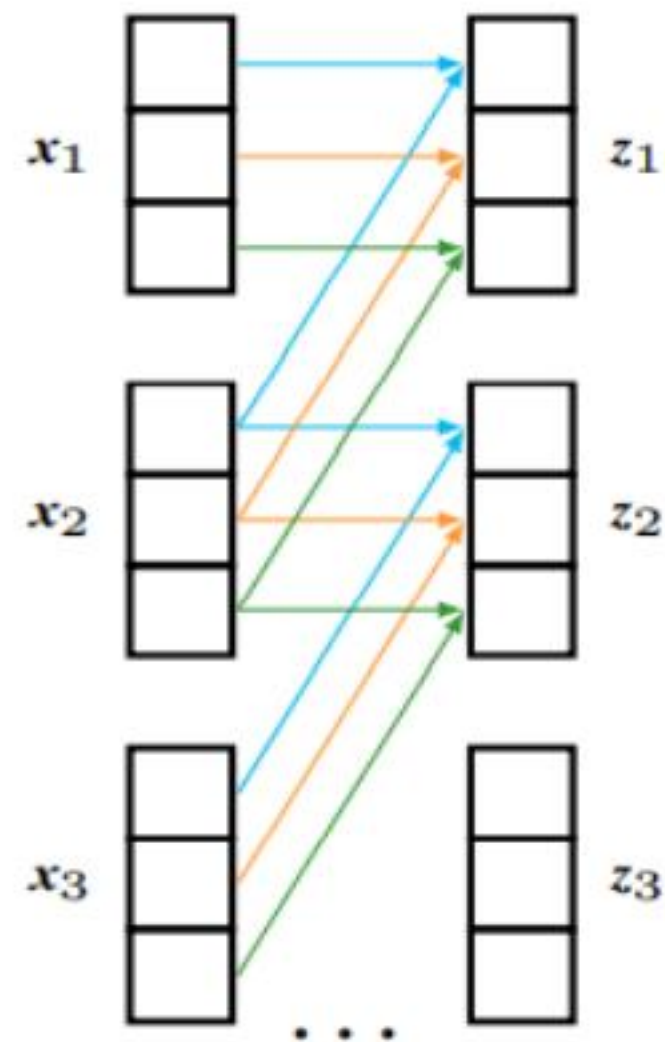
考虑不同词的不同特征



(a) 标准卷积

$$z_{i,n} = \sum_{o=1}^O \sum_{k=0}^{K-1} x_{i+k,o} W_{k,o,n}$$

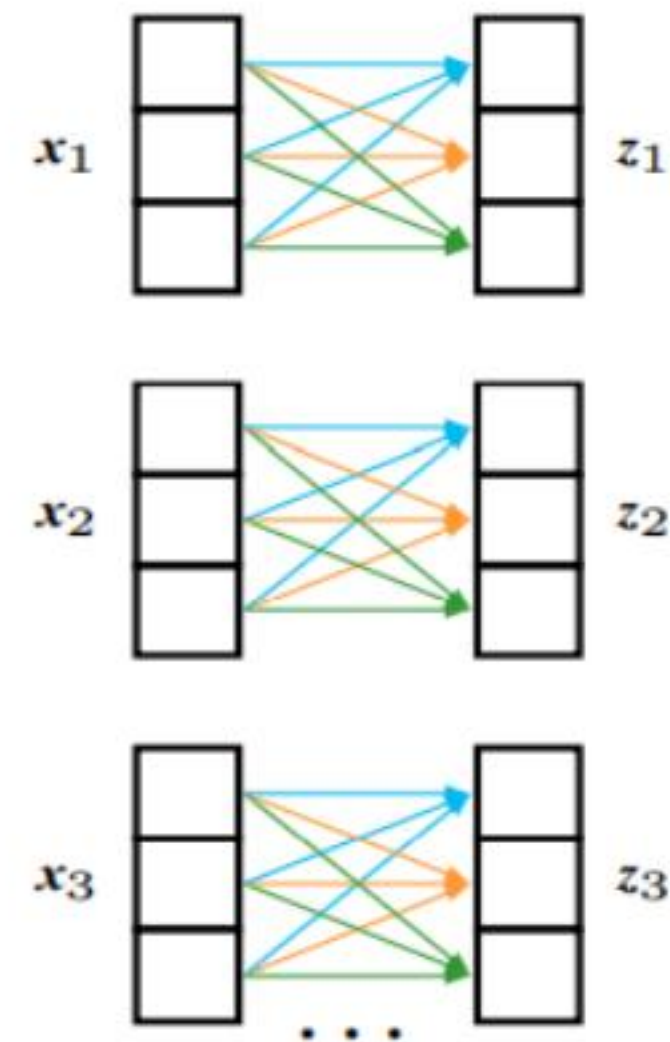
考虑不同词



(b) 深度卷积

$$z_{i,o} = \sum_{k=0}^{K-1} x_{i+k,o} W_{k,o}$$

考虑不同特征



(c) 逐点卷积

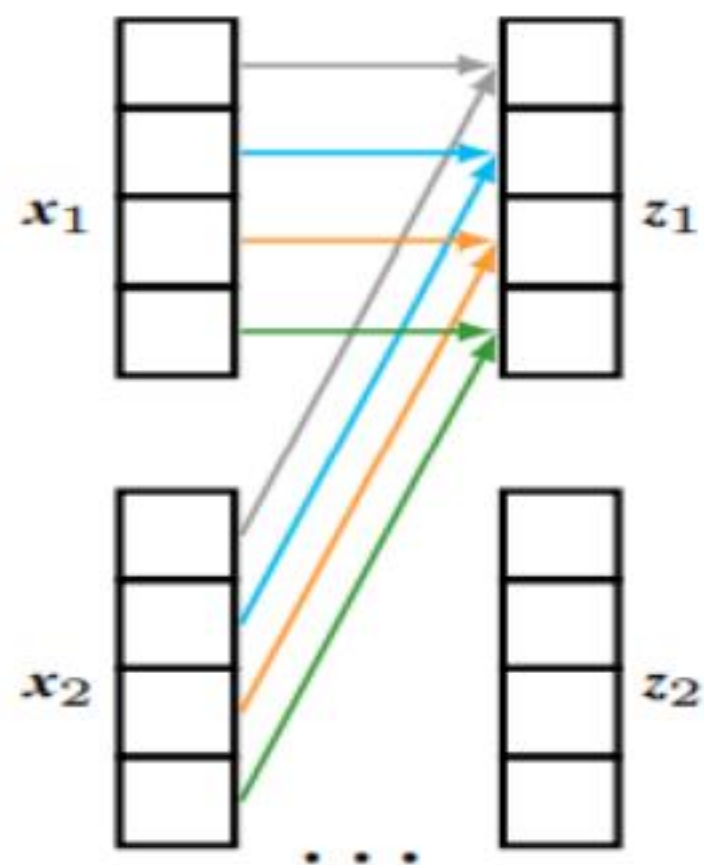
$$z_{i,n} = \sum_{o=1}^O x_{i,o} W_{o,n} = x_i W$$



# 3.卷积神经网络模型

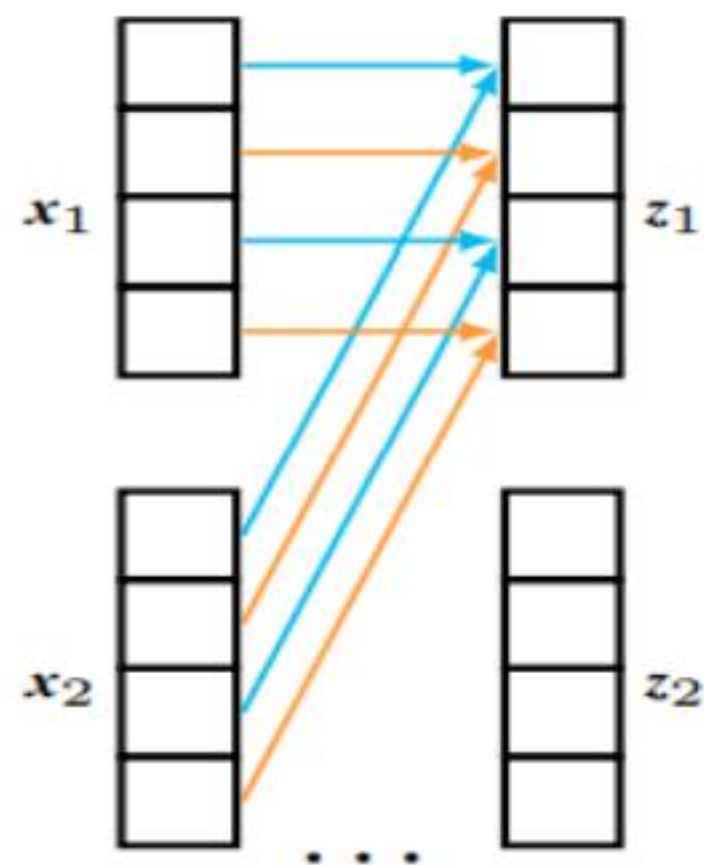
轻量卷积：不同通道共享卷积核参数，降低参数量。

动态卷积：卷积核参数根据输入动态生成，增强模型的表示能力。



深度卷积

$$z_{i,o} = \sum_{k=0}^{K-1} x_{i+k,o} W_{k,o}$$



轻量卷积

$$z_{i,o} = \sum_{k=0}^{K-1} x_{i+k,o} \text{Softmax}(W)_{k, [\frac{oa}{d}]}$$

Softmax实现对输出大小的限制

# 目 录

01

神经网络与语言建模

02

循环神经网络模型

03

卷积神经网络模型

04

自注意力神经网络模型

# 4.自注意力神经网络模型

自注意力机制：直接对不同位置单词之间的关系进行建模。

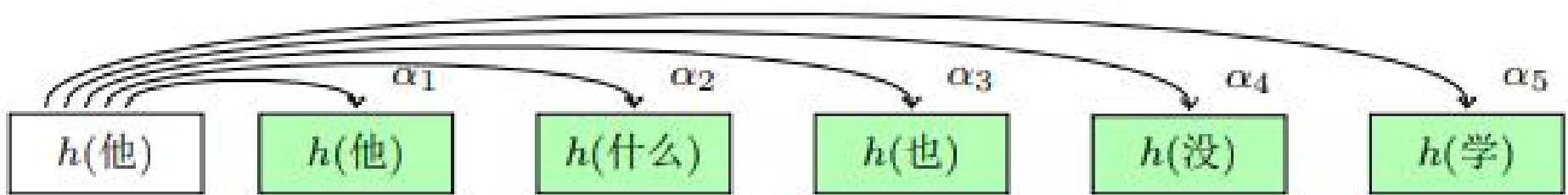
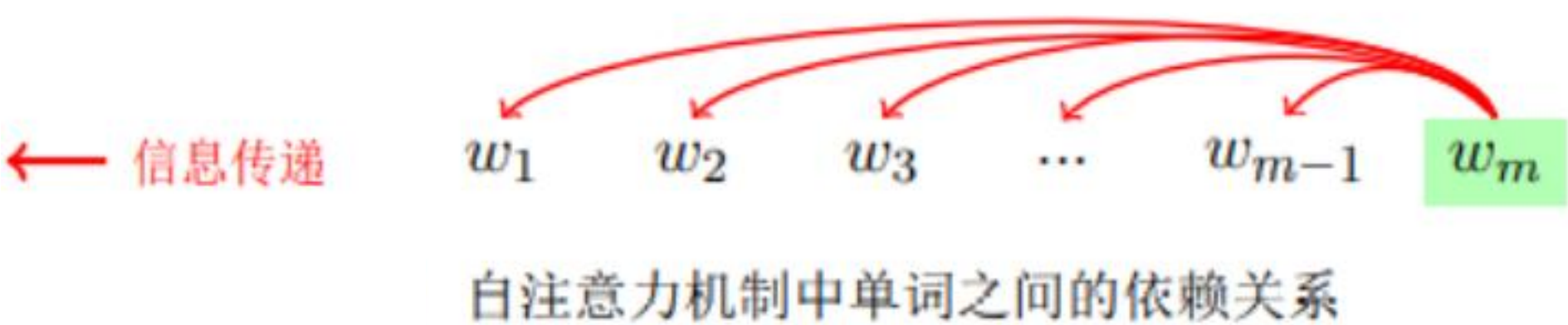
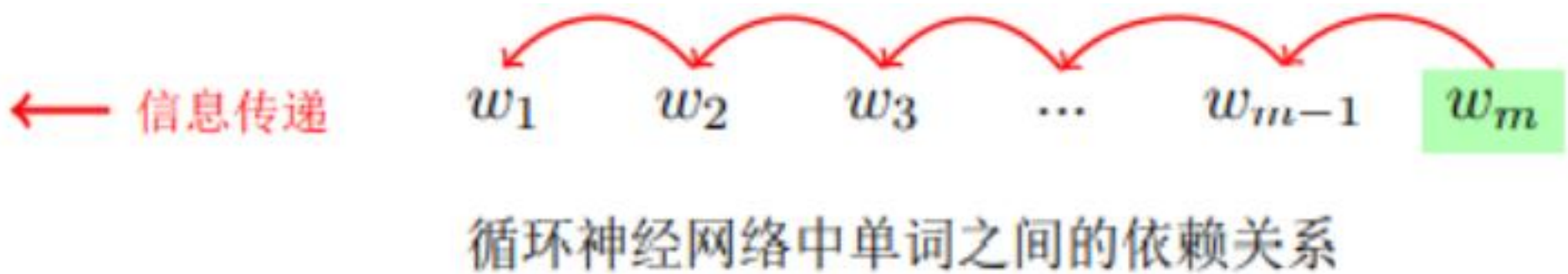


图 12.3 自注意力机制的计算实例

$$C_j = \sum_i a_{i,j} h_i$$

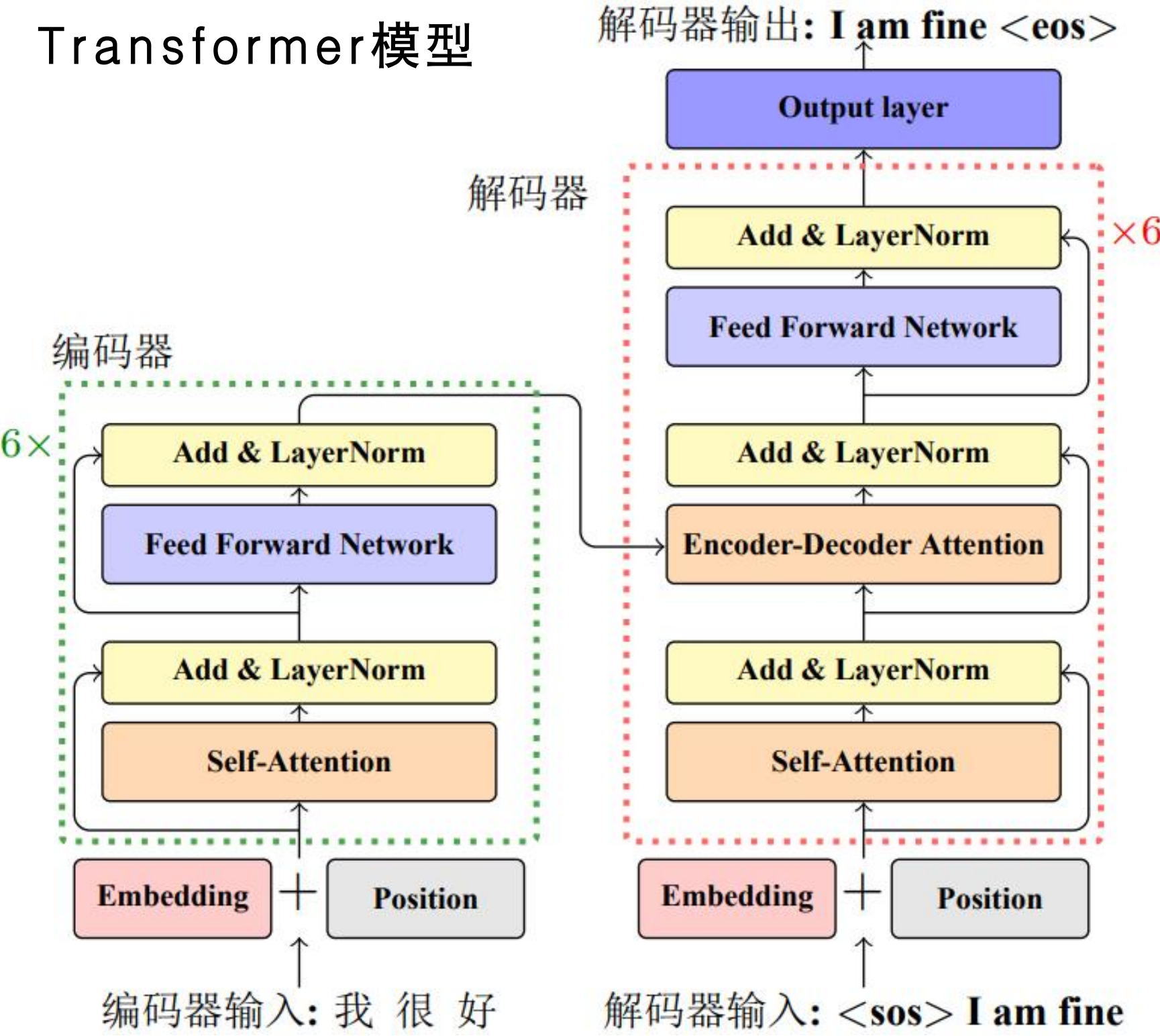
自注意力机制将目标位置的表示 $h_j$ 看作query，并将所有位置的表示看作key和value。query与key相关度用 $a_{i,j}$ 表示，value值为该位置的表示 $h_i$ 。

$$\tilde{h}(\text{他}) = \alpha_1 h(\text{他}) + \alpha_2 h(\text{什么}) + \alpha_3 h(\text{也}) + \alpha_4 h(\text{没}) + \alpha_5 h(\text{学})$$



# 4.自注意力神经网络模型

Transformer模型

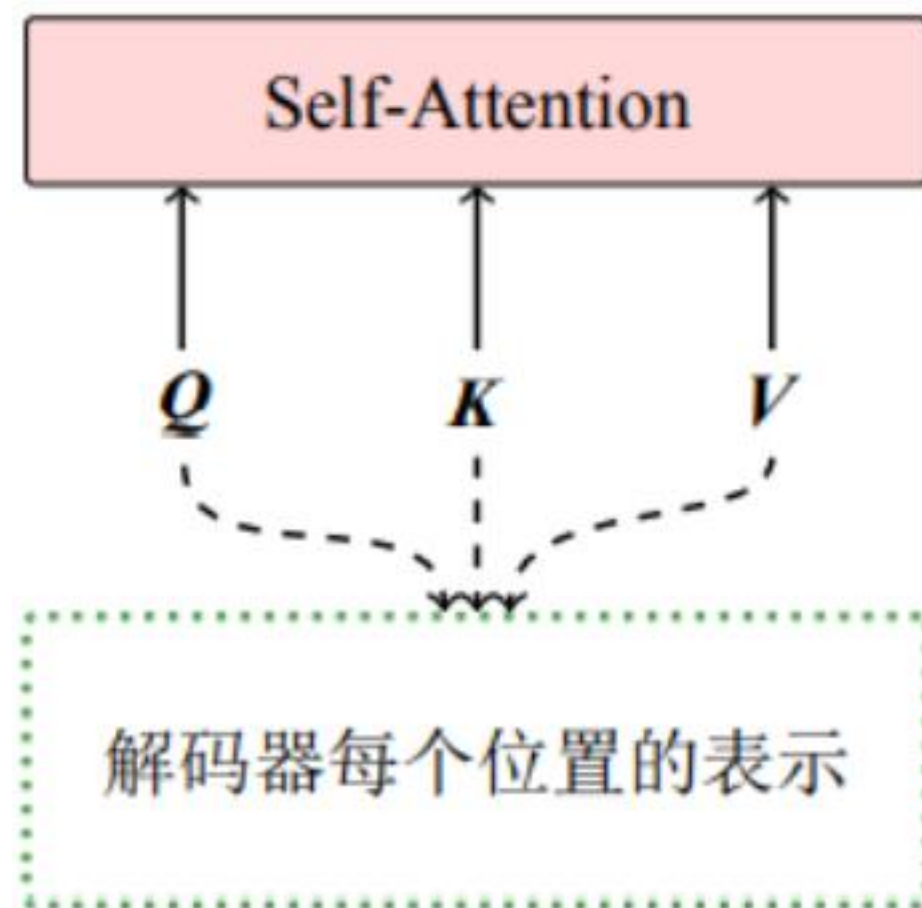


- ❖ **自注意力子层(Self-Attention Sub-layer):** 使用自注意力机制对输入的序列进行新的表示;
- ❖ **前馈神经网络子层(Feed-Forward Sub-layer):** 使用全连接的前馈神经网络对输入向量序列进行进一步变换;
- ❖ **残差连接(标记为“Add”):** 对于自注意力子层和前馈神经网络子层, 都有一个从输入直接到输出的额外连接, 也就是一个跨子层的直连。残差连接可以使深层网络的信息传递更为有效;
- ❖ **层标准化(Layer Normalization):** 自注意力子层和前馈神经网络子层进行最终输出之前, 会对输出的向量进行层标准化, 规范结果向量取值范围, 这样易于后面进一步的处理。
- ❖ **编码-解码注意力子层(Encoder-Decoder Attention Sub-layer):** 帮助模型使用源语言句子的表示信息生成目标语言不同位置的表示。仍基于自注意力机制。

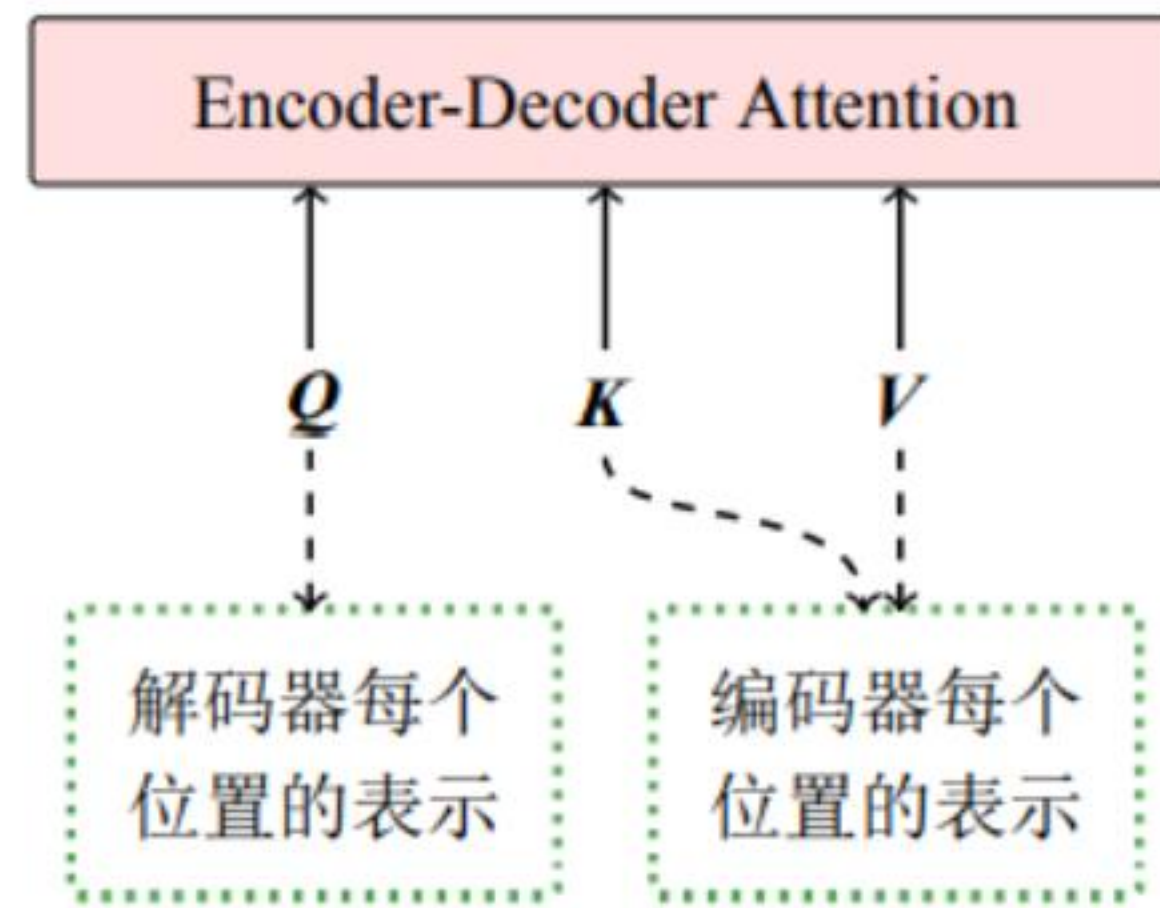


## 4. 自注意力神经网络模型

解码器中的自注意力子层与编码-解码注意力子层的输入区别



(a) Self-Attention 的输入

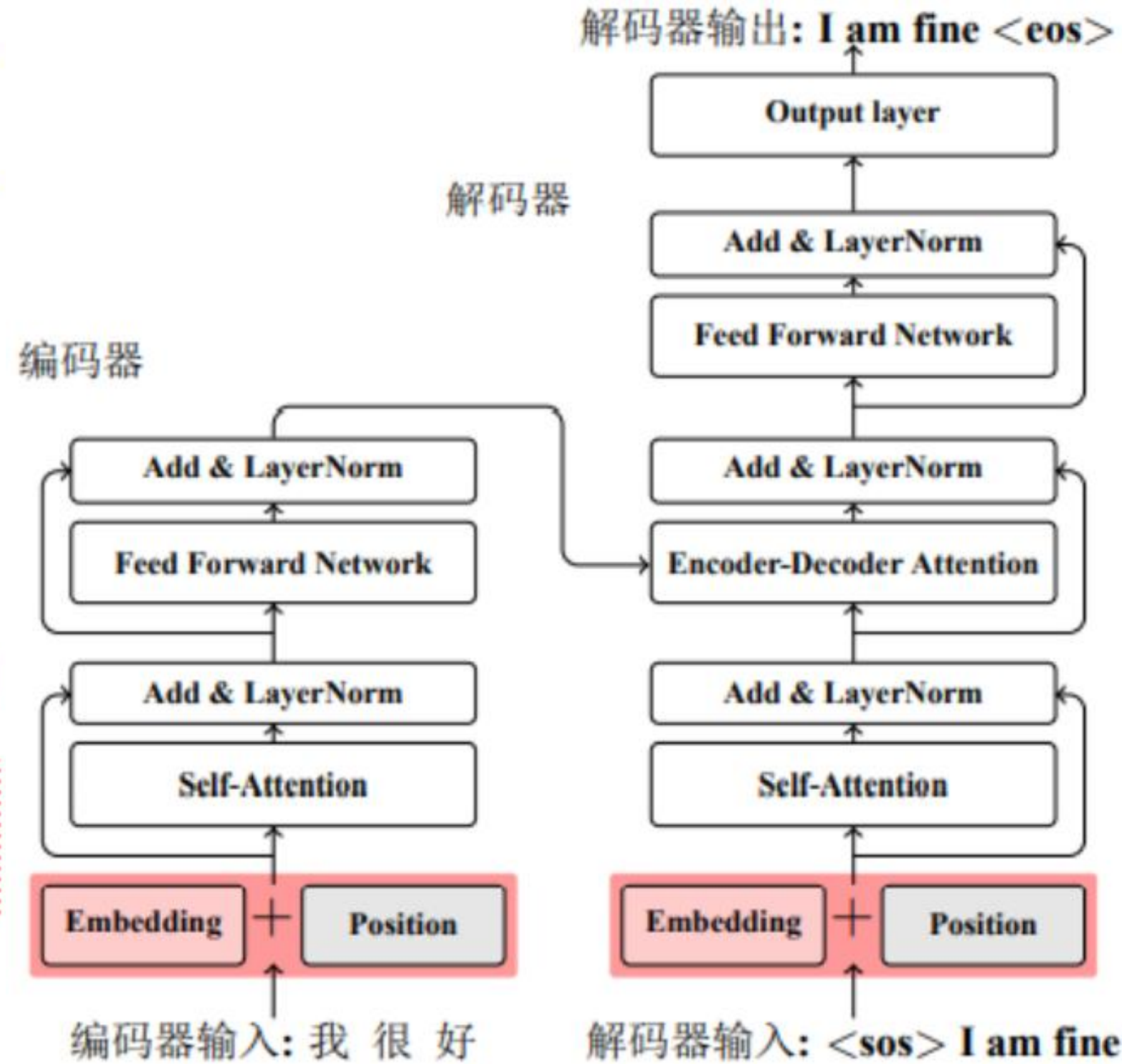
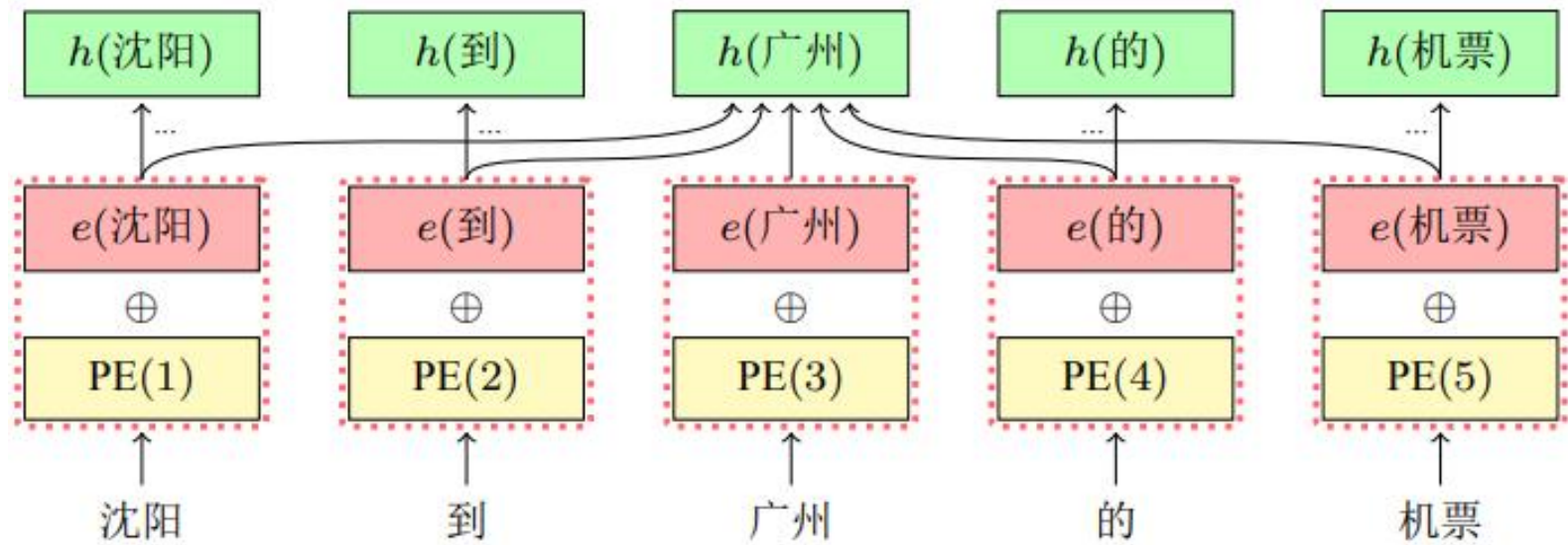


(b) Encoder-Decoder Attention 的输入

# 4.自注意力神经网络模型

**问题：** 直接对当前位置和序列中的任意位置建模，忽略了词之间的顺序关系。

引入**位置编码**，Transformer使用不同频率的正余弦函数。pos表示单词位置，i表示位置编码向量的第i维



# 4.自注意力神经网络模型

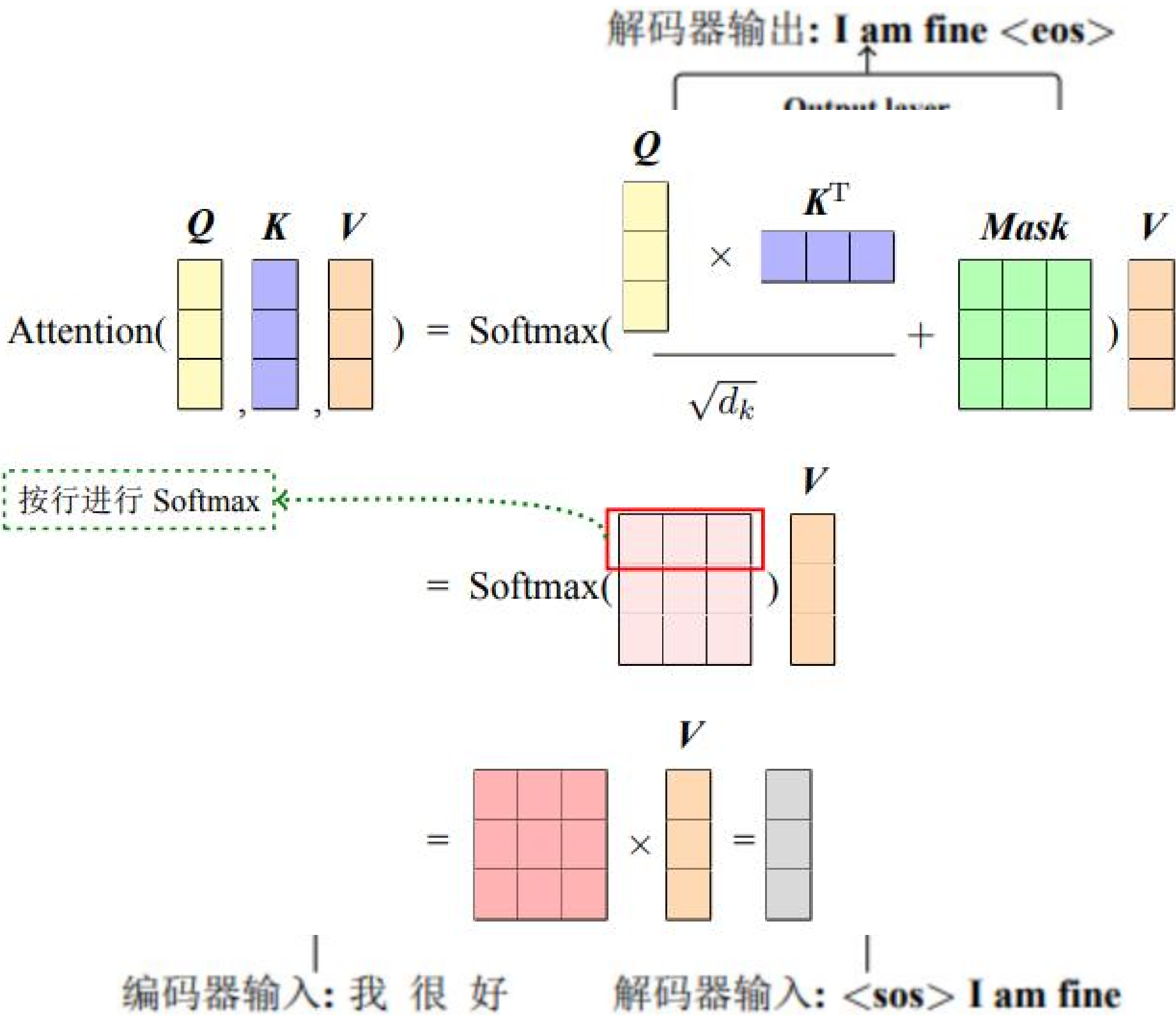
## 点乘注意力机制

query,key,value分别用Q 、 K 、 V表示， 维度为L × d,L为序列长度， d为输入向量维度。

$Attention(Q, K, V)$

$$= Softmax\left(\frac{QK^T}{\sqrt{d}} + Mask\right)V$$

$QK^T$  为L × L维的相关性矩阵；  $1/\sqrt{d}$  进行放缩减小方差； Mask为掩码矩阵， 用于屏蔽信息； Softmax函数在行维度上进行归一化操作。





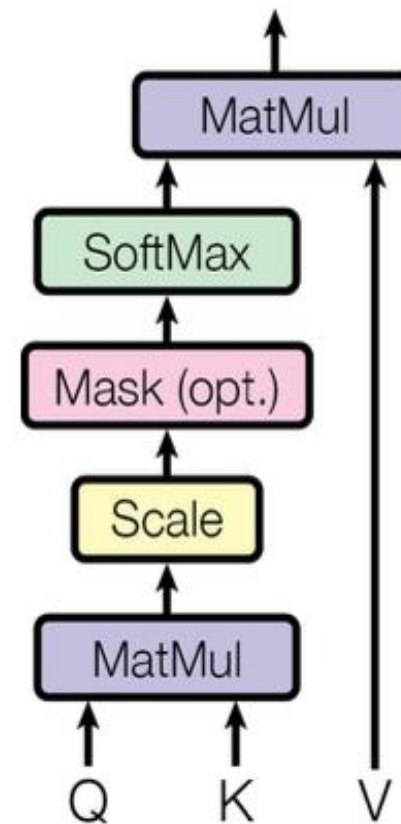
## 4. 自注意力神经网络模型

**多头注意力机制**：允许模型在不同的表示子空间学习，捕捉不同的信息。

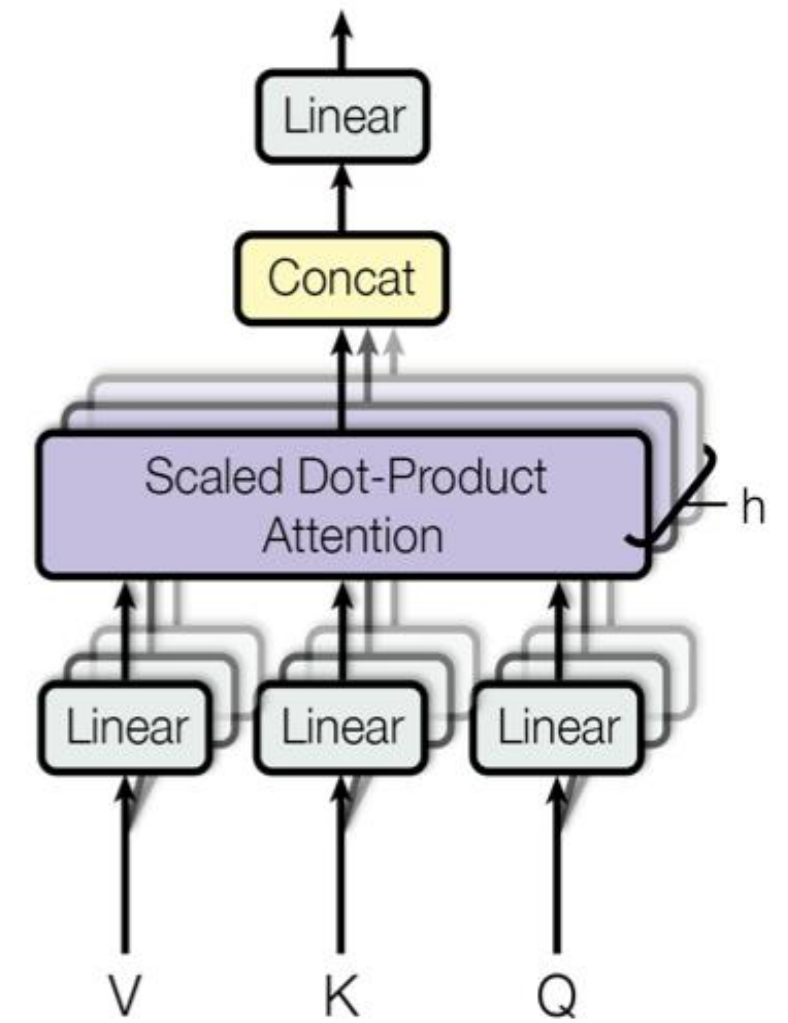
将  $Q$ 、 $K$ 、 $V$  按照隐藏层维度平均切分成多份，得到  $Q = \{Q_1, \dots, Q_h\}$ ， $K = \{K_1, \dots, K_h\}$ ， $V = \{V_1, \dots, V_h\}$ ，将每个切分单读进行注意力计算， $head_i = \text{Attention}(Q_i, K_i, V_i)$ 。

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W$

Scaled Dot-Product Attention



Multi-Head Attention



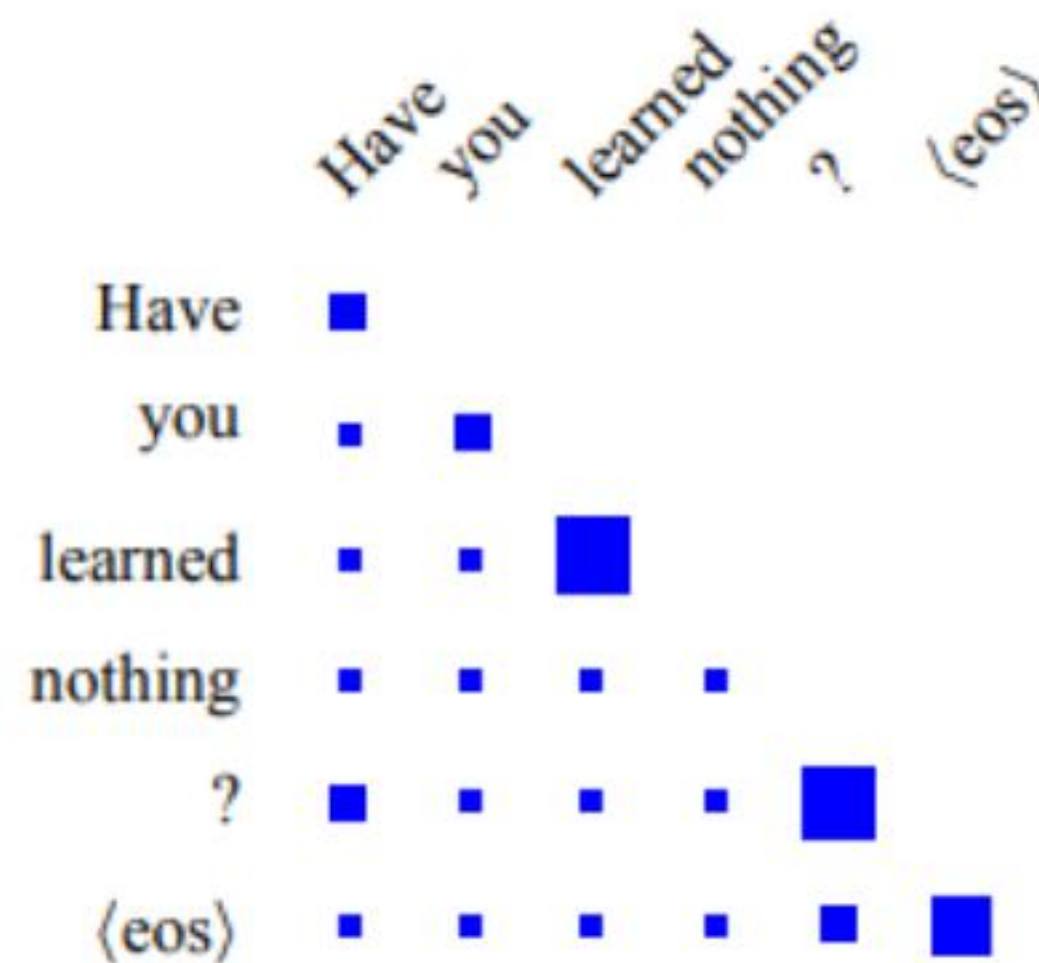
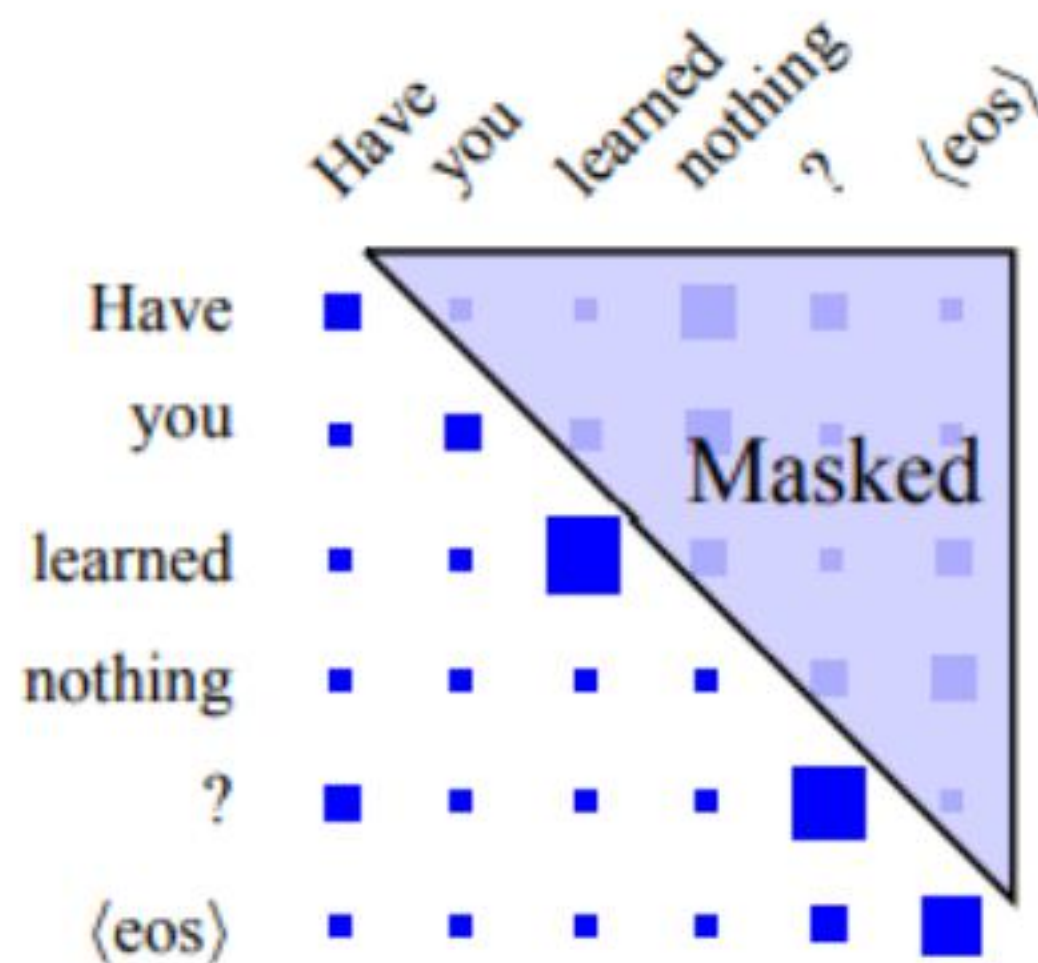


## 4.自注意力神经网络模型

**掩码矩阵**：在需要掩码的位置赋值负无穷 $-\infty$ ，其余位置为0，在Softmax归一化后，掩码位置权值近似为0。

**句长补全掩码**：在批量处理多个样本时，由于样本长度不一样，需要对较短样本后面填充0占位，在计算注意力时需要用掩码屏蔽。

**未来信息掩码**：在解码器预测时，预测是自左向右进行的，需要对未来信息进行掩码屏蔽。



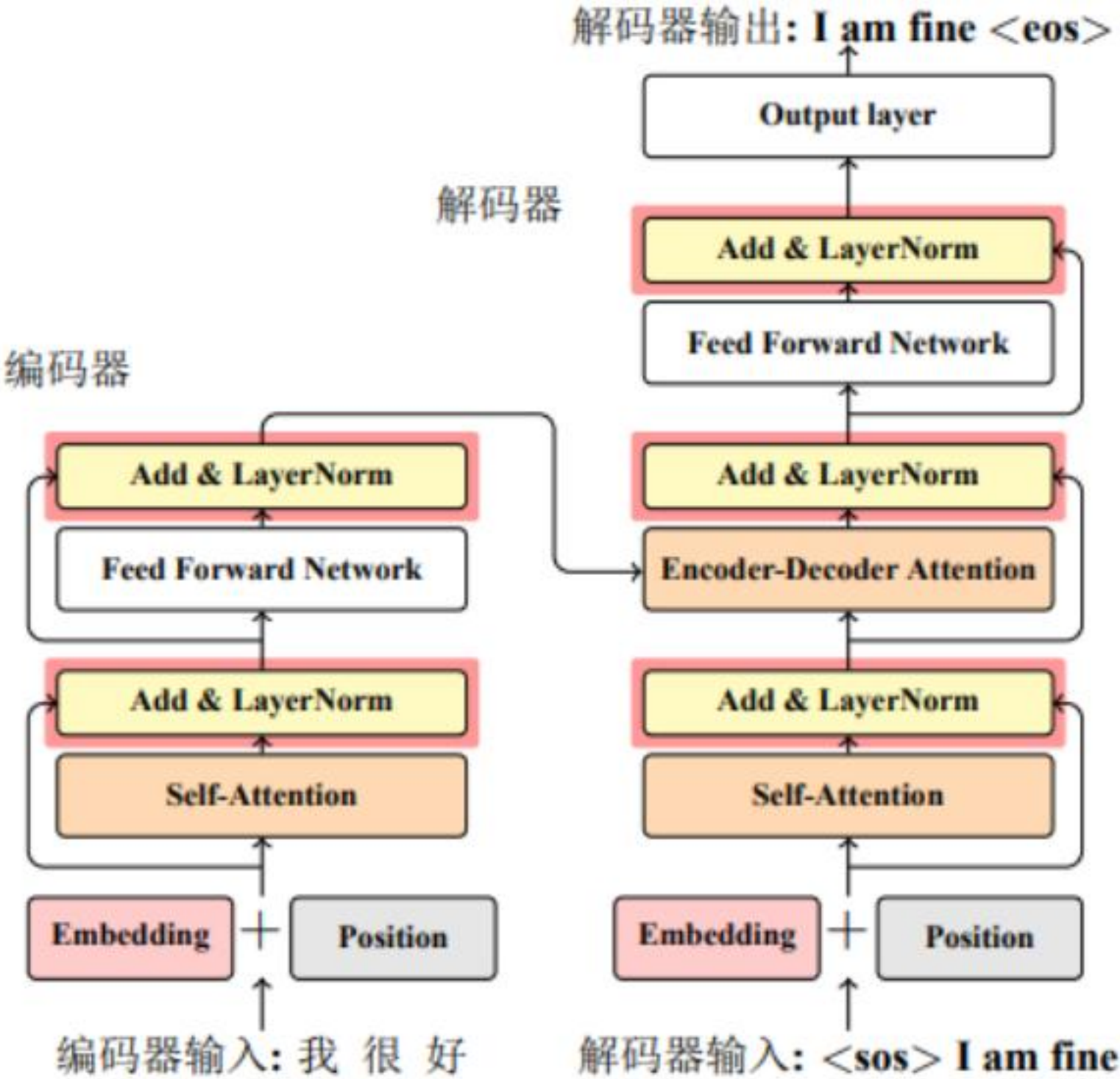
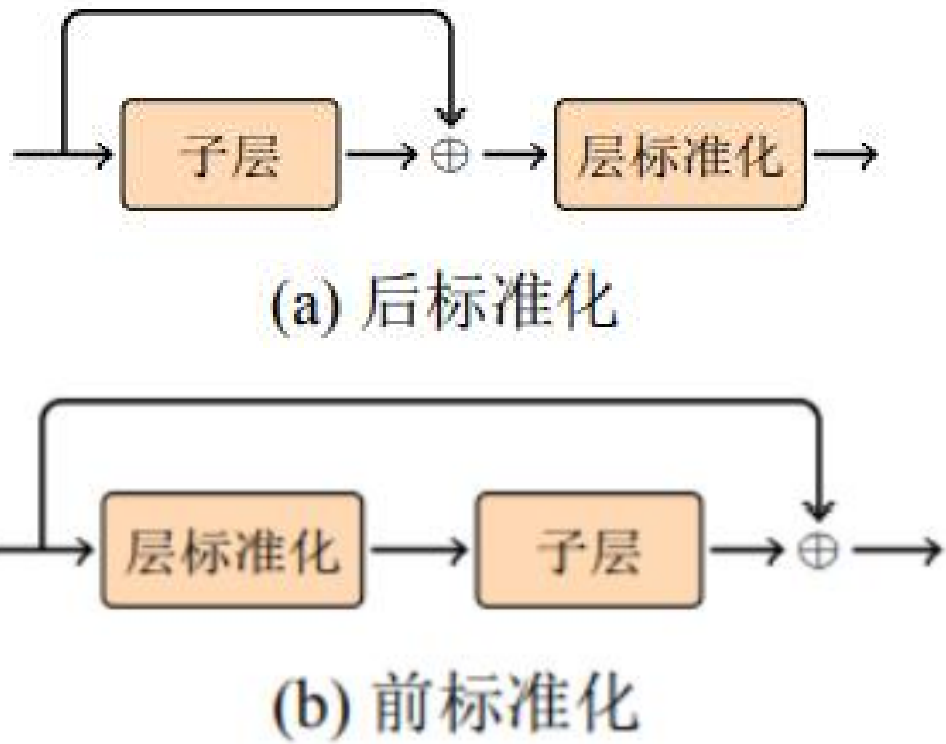
# 4.自注意力神经网络模型

**残差网络**：解决反向传播时，梯度爆炸或梯度消失问题。

$$x^{l+1} = F(x^l) + x^l$$

**层标准化**：解决引入残差连接，不同层的结果差异性很大，训练过程不稳定、训练时间较长问题。

$$LN(x) = g \cdot \frac{x - \mu}{\sigma} + b$$



## 4.自注意力神经网络模型

**前馈全连接网络子层**：将经过注意力计算后的表示映射到新的空间，有利于接下来的非线性变换等操作。

Transformer的全连接前馈神经网络包含两次线性变换和一次非线性变换。

$FFN(x)$

$$= \max(0, xW_1 + b_1)W_2 + b_2$$

